VOTR: Vector Orchestrated Tool Retrieval for Scalable Multi-Agent Systems

Amitahbh Kumar[1,2], Fethi Rabhi[1], Atharva Tendle[3], Debanjan Mahata[3], Shou Zhou[4], George Chrysostomou[4], Eason Wang[2]

[1] *University of New South Wales, Sydney, Australia*
[2] *Bloomberg, Sydney, Australia*
[3] *Bloomberg, New York, USA*
[4] *Bloomberg, London, UK*

Abstract

Large language model agents increasingly rely on the Model Context Protocol (MCP) to access external tools, yet injecting an entire tool catalogue into the model context is prohibitively expensive at scale. MCP-Zero demonstrated that embedding-based proactive retrieval can reduce token consumption by approximately 98% while retaining high accuracy, but its design is limited to offline research settings with a static index, fixed top-k and no live MCP integration. This paper presents VOTR, a production-deployable tool retrieval service that extends MCP-Zero with a hybrid retrieval pipeline fusing dense embeddings, BM25 and SPLADE-lite sparse expansion via weighted Reciprocal Rank Fusion; field-aware re-ranking over structured tool metadata; adaptive top-k selection gated by a calibrated non-conformity score; zero-downtime hot registration of new MCP servers; overlap-aware disambiguation and optional abstention guarding; compressed schema injection; and multi-turn session memory tracking. On a 500-query single-tool benchmark spanning 309 servers and 2,806 tools, VOTR achieves 96.8% top-1 accuracy with sub-350 ms median routing latency and an average context injection of approximately 230 tokens per routing decision. Multi-hop and multi-tool suites are aligned to the same evaluation counts (100 / 250 / 500 routing steps at small / medium / large): hop-level top-1 on the large catalogue reaches 96.8%, while strict *all-hop* success at k=1 is 90% (95% at k=3, 97% at k=5); an equivalence-aware label map recovers hop-level top-1 to 97.4% and all-hop success at k=1 to 92%. In low-cardinality deployments involving a single server ecosystem, VOTR reaches ceiling-level retrieval performance (100%) on 100-query Bloomberg, GitHub and Telegram one-server subsets. Codes are available at: https://github.com/iamAmiK/VOTR

Keywords: Model Context Protocol, tool retrieval, hybrid retrieval, LLM agents, agent systems

Contents

1 Introduction

1.1 Context and Motivation

The past two years have seen a rapid transformation in the way large language models interact with the external world. Where early systems relied on a handful of hand-wired function calls, modern agent frameworks provide LLMs with access to entire ecosystems of tools; file systems, code interpreters, web browsers, messaging platforms, financial data feeds and thousands of specialised SaaS APIs; all through standardised interfaces. The most prominent of these interfaces is the Model Context Protocol (MCP) [1], an open standard that defines a transport-agnostic JSON-RPC interface between an LLM host and any number of tool servers. By early 2025, more than three thousand community-built MCP servers had been published and the number continues to grow at an accelerating pace.

This abundance creates a fundamental engineering problem. A model's context window is finite and every token spent describing a tool is a token unavailable for reasoning about the user's actual request. Injecting the full catalogue of tool schemas into the prompt for every query is prohibitively expensive: a large scale deployment of 309 MCP servers exposing 2,806 tools would consume upwards of 300,000 tokens in schema descriptions alone or even a modest deployment of a single Github MCP server can exceed the context limits of most production models and severely degrading reasoning quality even when the window is large enough to accommodate the text [2–5].

The alternative; manually curating which tools to expose per agent or per task does not scale. New servers appear weekly, tool names overlap across providers and user intents are unpredictable. What is needed is an automated system that can in real time, select the small subset of tools most likely to be relevant to a given query, inject only those tools' schemas and do so with high accuracy, low latency and minimal token overhead. Recent MCP-oriented systems increasingly frame this as a structured retrieval problem rather than a pure prompting problem, including RAG-MCP, Dynamic ReAct, ToolScope and Tool-to-Agent Retrieval [4–7].

1.2 Problem Statement

MCP-Zero [2] provided the first convincing evidence that proactive tool retrieval is viable. By encoding both the user query and all tool descriptions with a shared embedding model and selecting top-k tools via cosine similarity, MCP-Zero reduced prompt token consumption by approximately 98% while maintaining 95.2% top-1 accuracy on a 308 server benchmark. This result established the principle that retrieval can replace full-catalogue injection.

However, MCP-Zero was designed as a research prototype and cannot be deployed in production today. Its tool catalogue is a frozen 326 MB JSON snapshot; adding or removing a single MCP server requires a full manual rebuild of the entire file. The retrieval pipeline is disconnected from actual MCP server transports as it operates on pre-embedded static data and cannot issue `tools/list` calls to live servers. Search is a flat brute-force cosine similarity over all server embeddings, which, while adequate for 308 servers, provides no structural mechanism for scaling

to larger catalogues. The number of tools returned is fixed at a static top-k regardless of retrieval confidence, meaning the system either injects too many irrelevant tools when it is confident (wasting tokens) or misses relevant ones when it is uncertain (reducing accuracy). Finally, there is no feedback loop: the system never learns which tools were actually selected by the model or whether the selected tool succeeded.

1.3 Research Contributions

This paper presents VOTR (Vector Orchestrated Tool Retrieval), a production-deployable tool retrieval service that addresses each of the limitations described above. VOTR is implemented as a FastAPI service that accepts structured routing requests and returns ranked, compressed tool candidates ready for LLM context injection.

The first contribution is a hybrid retrieval pipeline that fuses three complementary retrieval signals: dense embedding similarity (inherited from MCP-Zero's hierarchical server-then-tool scoring), BM25 keyword matching over tool documents and a lightweight SPLADE-inspired sparse expansion stage. These three ranked lists are merged via weighted Reciprocal Rank Fusion (RRF), combining the semantic generalisation of dense retrieval with the exact-match precision of sparse methods.

The second contribution is a field-aware re-ranking layer that decomposes the query into structured fields (server name, action verb, parameter constraints) and computes token-level overlap scores against corresponding tool metadata fields, using a weighted Sørensen–Dice coefficient to break ties that RRF alone cannot resolve.

The third contribution is an adaptive top-k selection mechanism gated by a calibrated non-conformity score inspired by conformal prediction. Rather than returning a fixed number of candidates, VOTR examines the score distribution of the top results and dynamically chooses whether to return one tool (when confidence is high), three tools (moderate confidence), or five tools (low confidence). The thresholds are calibrated on a held-out dataset to trade off candidate-set coverage and context budget; in the headline configuration this is used as a calibrated confidence policy rather than a formal coverage guarantee.

The fourth contribution is a zero-downtime dynamic registry that allows new MCP servers to be discovered and registered at runtime via stdio subprocess or HTTP/SSE endpoint; without rebuilding the index or restarting the service.

The fifth contribution comprises a set of robustness mechanisms: an overlap-aware disambiguation system that detects when multiple servers offer functionally identical tools and surfaces all alternatives; an optional abstention guard that suppresses low-quality candidates when no tool genuinely matches the query; and a regression test suite that ensures safety properties are preserved across code changes.

The sixth contribution is a compressed schema injection format. On our indexed tool corpus we count tokens with OpenAI's public `cl100k_base` byte-pair encoding (`tiktoken`): reproducing MCP-Zero's `<function>` JSON wrapper averages 93.5 tokens per tool, whereas VOTR's compact schema line averages 26.2 tokens per tool; a 72% reduction at the same retrieval depth (table 4).

The seventh contribution is a multi-turn session memory that tracks which tools have been

injected in prior conversations as session state, providing the foundation for future suppression of redundant re-injection and cumulative token savings proportional to conversation length.

1.4 Paper Organisation

The remainder of this paper is organised as follows. Section 2 reviews the Model Context Protocol, prior work on tool retrieval for LLM agents and the theoretical foundations of hybrid retrieval and conformal prediction. Section 3 presents the system architecture, API surface and data model. Section 4 details the full retrieval pipeline from dense scoring through RRF fusion to field-aware re-ranking. Section 5 describes the confidence-gated handoff policy and its conformal calibration. Section 6 covers the robustness and safety mechanisms. Section 7 presents the dynamic registry and hot registration. Section 8 discusses efficiency optimisations including schema compression and session memory. Section 9 reports comprehensive evaluation results across functional correctness (including scaled single-hop, multi-hop and multi-tool suites), ablation studies, confidence calibration, efficiency, multi-hop scaling, dynamic registration, negative controls and the LiveMCPBench benchmark. Section 10 analyses findings and limitations. Section 11 concludes.

2 Background and Related Work

2.1 The Model Context Protocol

The Model Context Protocol [1] defines a standardised JSON-RPC interface between an LLM host (the "client") and one or more tool servers. Each server exposes a `tools/list` endpoint that returns a structured catalogue of callable operations, where each tool is described by a name, a natural-language description and a JSON-Schema specification of its input parameters. Communication occurs over one of two transport modes: stdio pipes (for locally spawned processes) or HTTP with Server-Sent Events (SSE) for remote endpoints. The protocol also defines lifecycle methods (`initialize`, `ping`, `shutdown`) and a notification channel for capability updates.

The significance of MCP lies in its role as a universal adapter layer. Before MCP, each agent framework implemented its own bespoke tool interface, creating fragmentation and duplication of effort. With a shared protocol, any server can be used by any compatible agent and the tool ecosystem can grow independently of any single framework. This decoupling, however, amplifies the retrieval challenge: as the number of available servers grows, the cost of presenting all tools to the model becomes untenable.

2.2 Tool Retrieval for Large Language Model Agents

The problem of selecting which tools to present to an LLM has evolved through several stages. Early function-calling systems such as the initial GPT-4 release [8] supported only a small, fixed set of tools defined at prompt construction time. ToolBench [3] scaled to 16,000 APIs by organising them into a hierarchical category tree and using BM25 retrieval over API documentation to

select relevant candidates, but the approach suffered from high latency and token overhead at scale. ToolkenGPT [9] proposed embedding tools as special tokens in the model's vocabulary, eliminating the need for schema injection entirely, but at the cost of model modification and retraining. The tool-manipulation capabilities of open-source models were further examined in [**?**], highlighting persistent tool-use reliability gaps under realistic prompting settings. The Retrieval-Augmented Generation (RAG) paradigm [10] provided a more general framework: encode queries and documents with a shared embedding model, retrieve the most relevant documents and inject them into the prompt. MCP-Zero adapted this paradigm specifically for tool retrieval in the MCP ecosystem. Subsequent work has further explored MCP-specific retrieval under prompt-budget constraints, including RAG-MCP and Dynamic ReAct [4**?** , 5], while recent tool-to-agent routing formulations explicitly connect tool-level and agent-level retrieval spaces [7].

2.3 MCP-Zero: The Proactive Retrieval Baseline

MCP-Zero [2] introduced the concept of *proactive* tool retrieval, in which the agent infrastructure selects relevant tools *before* the model is invoked, rather than relying on the model itself to parse a long list of tools in its context. The system operates in two stages. First, the user query is embedded using a shared embedding model (OpenAI `text-embedding-3-large`) and cosine similarity is computed against pre-embedded server description and summary vectors. The top-N servers (typically N=5) are selected. Second, tool embeddings within those servers are scored against the query embedding and the top-k tools are returned.

On a benchmark of 308 servers and 2,797 tools constructed from a comprehensive crawl of the MCP ecosystem, MCP-Zero achieved 95.2% top-1 accuracy while reducing the number of tokens injected into the model's context by approximately 98% compared to full-catalogue injection. These results established a strong baseline for the proactive retrieval paradigm.

However, MCP-Zero's design choices reflect its research-prototype origins. The entire tool catalogue is stored as a single 326 MB JSON file (`mcp_tools_with_embedding.json`), pre-embedded offline. There is no mechanism for adding or removing servers without regenerating this file. Search is a brute-force matrix multiplication over all server embeddings, which is efficient at 308 servers but provides no indexing structure for larger catalogues. The top-k parameter is fixed, offering no way to adapt the number of returned tools to the confidence of the retrieval. There is no integration with live MCP transports, no feedback loop and no handling of edge cases such as overlapping capabilities across servers or queries that match no tool in the catalogue.

2.4 Sparse, Dense and Hybrid Retrieval Methods

Information retrieval has long distinguished between sparse (lexical) and dense (semantic) methods and the combination of the two has emerged as a robust best practice.

BM25 [11], the most widely deployed sparse retrieval function, scores documents by the weighted frequency of query terms, accounting for document length and corpus statistics via its k_1 and b parameters. BM25 excels at exact lexical matching: when a user mentions a specific tool name (e.g., `search_repositories`) or server name (e.g., "GitHub"), BM25 retrieves the

correct document with high precision. Its weakness is that it has no mechanism for semantic generalisation; paraphrases and synonyms are invisible to it.

Dense retrieval methods, exemplified by DPR [12], encode both queries and documents into a shared dense vector space and compute relevance via dot product or cosine similarity. These methods excel at semantic matching; a query about "finding code repositories" will correctly match the GitHub server even if the word "GitHub" never appears in the query. However it can struggle with rare or domain-specific identifiers that were underrepresented in the embedding model's training data.

SPLADE [13] bridges the gap by learning sparse representations that expand both queries and documents with related terms via a log-saturation activation over transformer token predictions. The resulting sparse vectors capture semantic relationships while remaining interpretable and efficient to index. Our SPLADE-lite variant approximates this expansion using TF-IDF with sublinear term-frequency scaling and bigram features, avoiding the need for a dedicated SPLADE model while still capturing short multi-word phrases and compound terms.

Reciprocal Rank Fusion (RRF) [14] provides a simple yet effective mechanism for combining multiple ranked lists without requiring score normalisation. For each item, the RRF score is the sum of reciprocal ranks across all input lists, smoothed by a constant k:

$$\mathrm{RRF}(j) = \sum_{\ell=1}^{L} \frac{w_\ell}{k + r_\ell(j)} \tag{1}$$

where $r_\ell(j)$ is the rank of item j in list ℓ and w_ℓ is an optional per-list weight. RRF is robust to score-scale mismatches between sparse and dense retrievers, and prior IR work reports that reciprocal-rank fusion is a strong rank-aggregation baseline that can outperform individual rankers [14].

Graph-structured retrieval has also emerged as a complementary direction for large tool ecosystems, where relationships between tools are explicitly modeled rather than treated as independent documents, as seen in GraphRAG-MCP [15]. In parallel, ToolScope combines automatic tool merging with hybrid retrieval to reduce tool overlap before selection [6]. These trends reinforce the importance of integrating structural signals with sparse-dense fusion in scalable tool retrieval systems.

2.5 Conformal Prediction and Calibrated Uncertainty

Conformal prediction [16] is a framework for constructing prediction sets with finite-sample coverage guarantees. Given a calibration set, one computes a *non-conformity score* for each example that measures how "unusual" the true label is under the model's output. At test time, all labels whose non-conformity score does not exceed a calibrated threshold are included in the prediction set, guaranteeing that the true label is included with probability at least $1-\alpha$ for a user-chosen significance level α.

In the context of tool retrieval, we adapt this framework to control the number of candidate tools surfaced per query. The non-conformity score is derived from the score gap and ratio between the top-ranked candidates and the calibrated thresholds determine whether one, three or

five tools are returned. Because the final VOTR policy also includes heuristic structural features, sparse-dense fusion and optional abstention rules, we use this machinery as a non-conformity-inspired calibration mechanism rather than claiming distribution-free conformal coverage for the deployed headline configuration. It nevertheless provides a principled way to trade context size (fewer tokens) against empirical candidate coverage (higher probability of including the correct tool), complementing recent RL-based retrieval optimization approaches such as Q-RAG [17].

At a broader systems level, routing in multi-agent settings has also been studied through graph-supervised coordination mechanisms, such as AgentRouter [18] and surveyed under unified agent workflow taxonomies [19]. VOTR focuses on tool-layer retrieval and schema-efficient routing in MCP ecosystems, but these adjacent lines of work motivate future integrations between tool retrieval and agent-level coordination.

3 System Architecture

3.1 Architectural Overview

VOTR is structured as a three-tier service, illustrated in fig. 1. The topmost tier is the agent or client application, which submits a routing request containing two natural-language fields: a *server intent* describing the domain or capability required (e.g., "GitHub repositories and API") and a *tool intent* describing the specific operation sought (e.g., "search for repositories matching a query string").

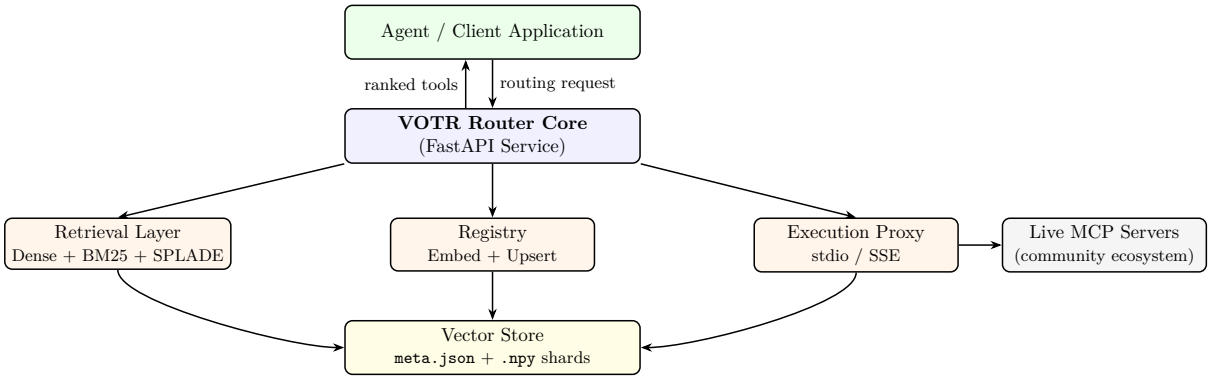

Figure 1: High-level architecture of VOTR. The Router Core orchestrates retrieval, re-ranking, confidence estimation and schema compression. The Registry handles dynamic server registration. The Execution Proxy optionally routes confirmed tool calls to live MCP server transports.

The middle tier is the Router Core, implemented as a FastAPI application. Upon receiving a request, the Router Core orchestrates the full retrieval pipeline: it embeds the query pair using the OpenAI embedding API, performs hierarchical dense retrieval against the in-memory ToolIndex, runs BM25 and SPLADE-lite sparse retrieval in parallel, fuses the three ranked lists via weighted RRF, applies field-aware re-ranking and intent-aware disambiguation, computes the non-conformity score and confidence tier, selects the adaptive top-k, compresses the tool schemas and returns a structured response.

The bottom tier comprises two sub-systems. The Registry maintains the live catalogue of

registered MCP servers, handles on-demand embedding of new servers and persists the updated index to disk. The Execution Proxy (optional) routes confirmed tool calls through to the appropriate MCP server transport, though the primary design focus of VOTR is on retrieval and ranking rather than execution.

The Vector Store is the shared persistence layer, consisting of `meta.json`; server metadata and tool definitions, and `.npy` shards; embedding matrices for server descriptions, server summaries and individual tools.

In addition to the core router service, our implementation includes a separate orchestrator layer used for end-to-end integration testing. This layer decomposes user requests into routing hops, calls VOTR's `/route` endpoint per hop, wraps routed tools into executable tool objects and runs a tool-calling agent to complete multi-step tasks against live MCP servers. In our reference implementation, this orchestration layer is built with LangChain's agent and structured-tool interfaces [20], while keeping VOTR itself framework-agnostic at the retrieval API boundary.

3.2 API Surface and Request Lifecycle

VOTR exposes six REST endpoints. The `GET /health` endpoint serves as a liveness probe, returning the status and current index directory path. The `POST /route` endpoint is the primary retrieval interface: it accepts a `RouteRequest` containing `server_intent`, `tool_intent` and an optional `session_id` and returns a `RouteResponse` containing a ranked list of `RoutedTool` objects, each with the tool's key, server name, RRF score, compressed schema line, full description and parameter schema. The response also includes the adaptive-k value, the top-1 and top-2 scores, the score gap, the confidence label (`high`, `medium` or `low`), the recommended handoff-k and optional overlap-ambiguity flags.

The `POST /register` endpoint accepts a pre-built `RegisteredServer` object and adds it to the live index. The `POST /register/discover` endpoint accepts a command and arguments for a stdio MCP server, spawns it as a subprocess, issues the MCP `initialize` handshake and `tools/list` call, normalises the response into a `RegisteredServer`, embeds it and upserts it into the index. The `POST /register/discover/sse` endpoint performs the same discovery over an HTTP/SSE endpoint. Finally, the `POST /session/clear` endpoint purges a session's tool history.

The request lifecycle for a typical `/route` call proceeds as follows. The server intent and tool intent are each embedded via the OpenAI embedding API, producing two 3072-dimensional vectors. The server intent embedding is used to score all registered servers via the hierarchical server scoring formula, selecting the top-N servers. Simultaneously, the concatenated query is passed to BM25 and SPLADE-lite retrievers. The three resulting ranked lists are fused via weighted RRF. The fused list is then re-ranked by the field-aware bonus and adjusted by intent-aware disambiguation. The handoff policy computes the non-conformity score and assigns a confidence tier. The overlap detector checks for cross-server capability collisions. The abstention guard checks for low-support candidates. Finally, the top-k tools are compressed and returned. Figure 2 summarises this end-to-end route lifecycle.

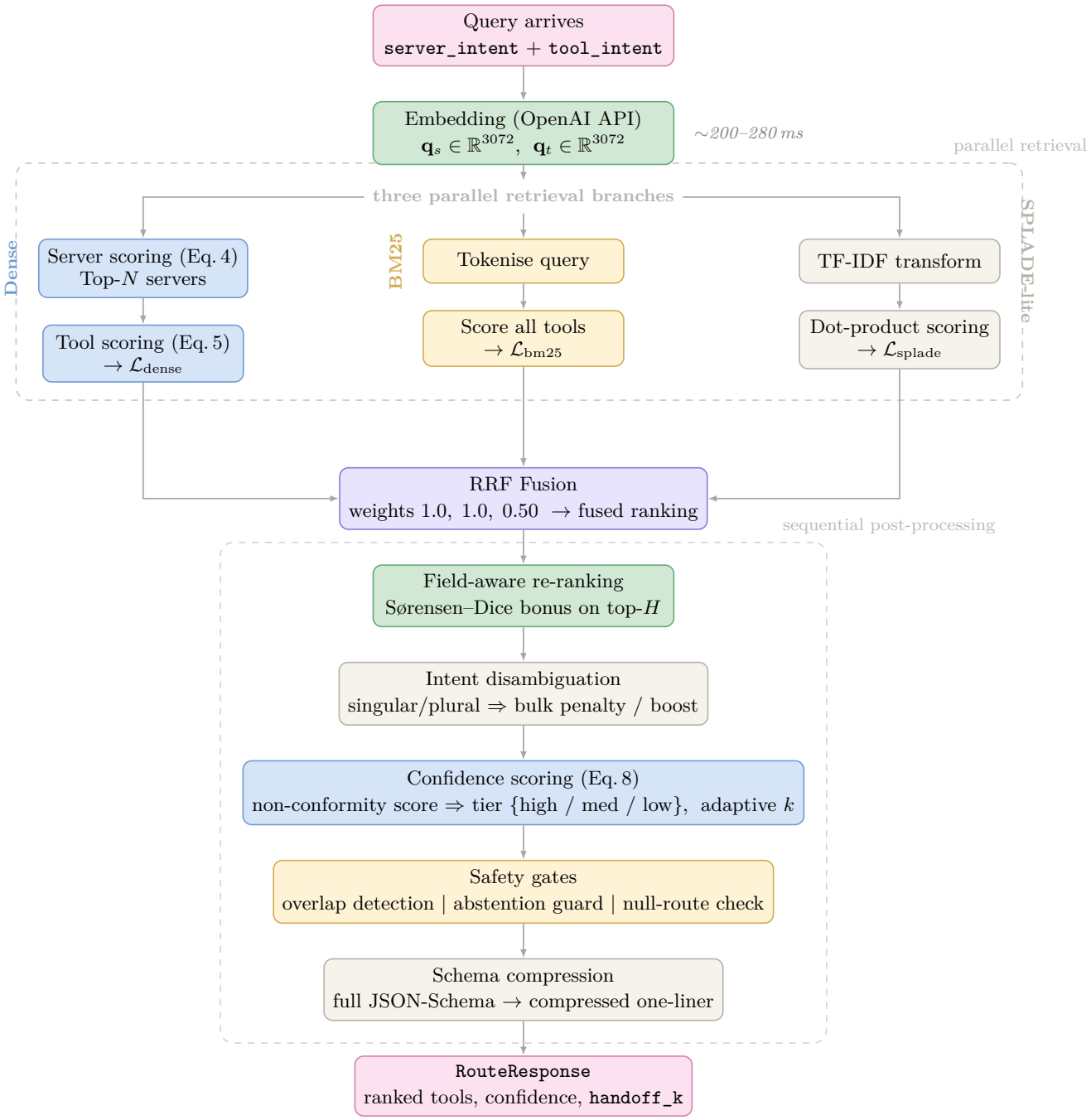

Figure 2: Retrieval pipeline for a single `/route` call. Embedding dominates latency (∼200–280 ms via OpenAI API); the three parallel retrieval branches and all downstream stages execute locally in <20 ms. Dashed containers mark the two main phases: *parallel retrieval* (Dense + BM25 + SPLADE-lite) and *sequential post-processing* (field-aware re-ranking through schema compression).

3.3 Data Model and Index Representation

The core data model is built around three Pydantic-validated types. A `RegisteredTool` contains a name (e.g., `search_repositories`), a natural-language description, a parameter dictionary mapping parameter names to their JSON-Schema types and optional usage statistics. A `RegisteredServer` aggregates a server name, description, summary (a short routing-oriented sentence), a list of `RegisteredTool` objects and a source tag indicating provenance. The `ToolIndex` holds all embedding matrices as L2-normalised NumPy arrays: server description embeddings ($S \times D$), server summary embeddings ($S \times D$), tool embeddings ($T \times D$), plus integer arrays mapping each tool row to its parent server index and local tool position. Here S is the number of servers, T is the total number of tools across all servers and D=3072 is the embedding dimensionality.

Table 1 compares the index formats of MCP-Zero and VOTR.

Table 1: Index format comparison: MCP-Zero vs. VOTR.

Aspect	**MCP-Zero**	**VOTR**
Storage format	Single 326 MB JSON file	`meta.json` + `.npy` shards
Rebuild strategy	Full manual rebuild	Incremental hot registration
Embedding storage	Inline in JSON	Separate binary arrays
Memory footprint	Entire file in RAM	Memory-mapped arrays
Parse method	Full JSON load	Streaming via `ijson` for build

4 Retrieval Pipeline

4.1 Hierarchical Dense Retrieval

The dense retrieval stage inherits and extends MCP-Zero's hierarchical server-then-tool scoring approach. Given a query pair consisting of a server intent embedding $\mathbf{q}_s \in \mathbb{R}^D$ and a tool intent embedding $\mathbf{q}_t \in \mathbb{R}^D$, the first step computes a composite server similarity score for each registered server i:

$$\sigma_i = \frac{w_{\max} \cdot \max\big(\cos(\mathbf{q}_s, \mathbf{e}^{(i)}_{\text{desc}}),\ \cos(\mathbf{q}_s, \mathbf{e}^{(i)}_{\text{sum}})\big) + w_{\text{mean}} \cdot \frac{1}{2}\big(\cos(\mathbf{q}_s, \mathbf{e}^{(i)}_{\text{desc}}) + \cos(\mathbf{q}_s, \mathbf{e}^{(i)}_{\text{sum}})\big)}{w_{\max} + w_{\text{mean}}} \tag{2}$$

where $\mathbf{e}^{(i)}_{\text{desc}}$ and $\mathbf{e}^{(i)}_{\text{sum}}$ are the L2-normalised description and summary embeddings for server i and $w_{\max}$ and w_{mean} are configurable blend weights. The default configuration uses $w_{\max}$=1.0 and w_{mean}=0.0, reducing the formula to the maximum of the two cosine similarities, but the mean component can be enabled for servers with highly informative summaries.

The top-N servers (default N=8) are selected by partitioned argsort. All tools belonging to these servers form the dense candidate pool $\mathcal{C}_{\text{dense}}$. Each tool $j \in \mathcal{C}_{\text{dense}}$ is then scored using a hierarchical combination that rewards both tool–query alignment and server–query alignment:

$$\tau_j = \sigma_{\pi(j)} \cdot t_j \cdot \max(\sigma_{\pi(j)},\ t_j) \tag{3}$$

where $t_j = \cos(\mathbf{q}_t, \mathbf{e}_j^{\text{tool}})$ is the tool embedding similarity and $\pi(j)$ maps tool row j to its parent server index. This multiplicative formulation ensures that a tool must achieve reasonable alignment on *both* the server and tool dimensions to receive a high score; a tool with excellent tool-level similarity but residing on an irrelevant server will be penalised and vice versa.

4.2 BM25 Sparse Retrieval

Each tool is represented as a single BM25 document that concatenates server context with tool-specific fields:

$$\text{doc}_j = s_{\text{name}}^{(\pi(j))} \parallel s_{\text{sum}}^{(\pi(j))} \parallel s_{\text{desc}}^{(\pi(j))} \parallel t_{\text{name}}^{(j)} \parallel t_{\text{desc}}^{(j)} \tag{4}$$

where $\parallel$ denotes string concatenation with whitespace separators. The query is formed by concatenating the server intent and tool intent. Tokenisation uses a shared normalisation expression across retrieval components: camelCase and snake_case are split into subwords, separators are normalised and lightweight plural stripping is applied (e.g., `repositories` → `repository`). Importantly, the token stream is intentionally *not* deduplicated so BM25 term-frequency signals are preserved.

The BM25Okapi implementation scores all tool documents against the query and produces a ranked list $\mathcal{L}_{\text{bm25}}$. BM25 provides a critical complementary signal to dense retrieval: when a user mentions a specific server or tool by name, BM25 retrieves the exact match instantly, whereas the embedding model may distribute similarity across semantically related but distinct entries.

4.3 SPLADE-Lite Lexical Expansion

Full SPLADE models [13] learn sparse representations via a masked language model head, requiring GPU inference at query time. For a production routing service where latency budgets are tight, this cost is undesirable. VOTR instead implements a lightweight approximation that we term SPLADE-lite.

The SPLADE-lite retriever constructs a TF-IDF matrix over all tool documents using unigram and bigram features with sublinear term-frequency scaling (tf → 1+log(tf)) and L2 normalisation. At query time, the query is transformed using the same vocabulary and the dot product against all document vectors yields a relevance score for each tool. While this approach does not perform the learned term expansion of full SPLADE, the bigram features capture common compound terms (e.g., “search repositories”, “send message”) and the sublinear scaling prevents high-frequency terms from dominating. The SPLADE-lite stage contributes a ranked list $\mathcal{L}_{\text{splade}}$ with a fusion weight of w_{splade}=0.50, compared to w_{dense}=1.0 and w_{bm25}=1.0 for the other two stages.

4.4 Reciprocal Rank Fusion

The three ranked lists $\mathcal{L}_{\text{dense}}$, $\mathcal{L}_{\text{bm25}}$ and $\mathcal{L}_{\text{splade}}$ are merged via weighted Reciprocal Rank Fusion. For each tool j that appears in at least one list, the fused score is computed as:

$$\mathrm{RRF}(j) = \sum_{\ell=1}^{3} \frac{w_\ell}{k + r_\ell(j)} \tag{5}$$

where $r_\ell(j)$ is the rank of tool j in list ℓ (with $r_\ell(j) = \infty$ if absent), k=60 is the standard rank-smoothing constant from the original RRF paper [14], and w_ℓ are the per-list weights.

Algorithm 1 Weighted Reciprocal Rank Fusion

Require: Ranked lists $\mathcal{L}_1, \ldots, \mathcal{L}_L$, weights $w_1, \ldots, w_L$, constant k
Ensure: Score map $S : \text{tool\_id} \to \mathbb{R}$

```
S ← ∅
for ℓ = 1 to L do
    for r = 1 to |L_ℓ| do
        j ← L_ℓ[r]
        S[j] += w_ℓ/(k + r)
return S
```

Algorithm 1 iterates over each ranked list, accumulating the weighted reciprocal rank contribution for each tool. The output is a dictionary mapping tool identifiers to fused scores, which is then sorted in descending order to produce the final merged ranking.

4.5 Field-Aware Re-ranking

After RRF fusion, the top-H candidates (default H=24) within a score window of δ_F=0.003 from the highest-scoring tool receive a field-aware re-ranking bonus. The purpose of this stage is to resolve ambiguities that the global retrieval signals cannot distinguish; for example, when two tools have similar descriptions but one matches the user's explicitly named server and the other does not or when the user's query contains a parameter name that aligns with one tool's parameter schema but not another's.

The query is first decomposed into three structured fields: a *server query* (derived from the server intent), an *action query* (the verb-oriented portion of the tool intent) and a *constraint query* (parameter-related terms extracted from the tool intent). If the server intent contains a known server name (e.g., "Bloomberg BLPAPI"), this is identified as an *explicit server name* and used for a direct matching bonus.

For each candidate tool j, the field-aware bonus is computed as:

$$B_j = \sum_{f \in \mathcal{F}} w_f \cdot \mathrm{Dice}(Q_f, T_j^f) + w_{\text{expl}} \cdot \mathbf{1}[\hat{s} = s_j] \tag{6}$$

where $\mathcal{F} = \{\text{server\_name}, \text{server\_summary}, \text{tool\_name}, \text{tool\_desc}, \text{params}\}$ is the set of metadata fields, $\mathrm{Dice}(a, b) = 2|A \cap B|/(|A| + |B|)$ is the token-level Sørensen–Dice coefficient computed on camelCase-split, underscore-normalised, plural-stripped tokens, Q_f is the corresponding query field, T_j^f is the candidate's metadata field and w_{expl}=0.22 is the bonus when an explicitly named server $\hat{s}$ matches the candidate's server s_j.

The final adjusted score for each candidate in the re-ranking window is

$$\hat{\tau}_j = \mathrm{RRF}(j) + \lambda \cdot B_j + \mu \cdot \max\!\big(0, \cos(\mathbf{q}_t, \mathbf{e}_j)\big),$$

where λ=0.0015 scales the lexical field bonus and $\mu=\lambda/2$ is a *dense margin recovery* term that re-injects a fraction of exact dense cosine similarity to break near-ties produced by RRF compression.

Table 2 lists the field weights. The tool name receives the highest weight because exact function-name matches are the strongest indicator of relevance in MCP tool routing.

Table 2: Field-aware re-ranking weights.

Field	**Weight**
Tool name	0.35
Server name	0.18
Tool description	0.18
Parameters	0.12
Server summary	0.08
Explicit server boost	0.22

4.6 Intent-Aware Disambiguation

A lightweight intent classifier analyses the tool intent string to detect two patterns that require special handling. The first pattern is singular destructive intent, identified by the presence of action verbs such as “delete”, “remove”, “clear”, or “destroy” without plural qualifiers. When this pattern is detected, tools whose names or descriptions contain bulk-operation indicators (“batch”, “bulk”, “all”, “multiple”) are penalised by a factor of 0.82, reducing the risk that a high-risk bulk operation appears as the top-1 result when the user intends a targeted single-item action.

The second pattern is plural or batch intent, identified by the presence of words such as “all”, “batch”, “every”, or “multiple” in the tool intent. In this case, bulk-capable tools receive a mild boost factor of 1.08, reflecting the user’s likely preference for batch operations. These adjustments are small by design: they serve as tiebreakers rather than overrides and their effect is most pronounced when two tools differ primarily in their single-vs-batch semantics.

5 Confidence-Gated Handoff Policy

The core insight behind the handoff policy is that the VOTR retrieval system is a candidate generator, not a final decision maker. In production, the downstream LLM receives the candidate tools and selects which one to call. Forcing a deterministic top-1 decision at the router level can amplify near-tie mistakes: when two tools are scored within a fraction of a percent of each other, the model should be given both and allowed to disambiguate using its richer understanding of the conversation context. Conversely, when the router is highly confident in its top result, injecting additional candidates wastes tokens and may confuse the model.

5.1 Adaptive Top-K Selection

The adaptive-k algorithm examines the score distribution of the top candidates after re-ranking and chooses how many to surface. The algorithm is deliberately simple: it inspects only the gap between the top-1 and top-2 scores against a single configurable threshold δ_c (default δ_c=0.12).

Algorithm 2 uses three branches: if the gap exceeds δ_c there is a clear winner and only the minimum number of tools (default 1) is returned; if the gap falls between $0.5 \cdot \delta_c$ and δ_c, a moderate number (3) is returned; and if the gap is smaller, the maximum configured number (default 8) is returned.

This formulation explicitly trades precision for recall under ambiguity, ensuring the downstream LLM retains access to the correct tool even when retrieval signals are noisy. Conversely, aggressively pruning to k

when the retriever is confident directly minimizes the number of tool schemas injected into the LLM context, reducing token overhead.

Algorithm 2 Adaptive Top-K Selection

Require: Sorted descending scores $[s_1 \geq s_2 \geq \cdots \geq s_n]$, confidence gap δ_c, bounds $k_{\min}$, $k_{\max}$
Ensure: Number of tools k to return

if $n = 0$ **then return** $k_{\min}$
if $n = 1$ **then return** 1
gap $\leftarrow s_1 - s_2$
if gap $\geq \delta_c$ **then**
    return $k_{\min}$ ▷ Clear winner
else if gap $\geq 0.5 \cdot \delta_c$ **then**
    return $\min(3, n, k_{\max})$ ▷ Moderate confidence
else
    return $\min(k_{\max}, n)$ ▷ Ambiguous

5.2 Non-Conformity Scoring

For the calibrated handoff policy, VOTR computes a non-conformity-inspired score from the top-2 adjusted scores and any structural signals in the query:

$$\mathrm{nc}(q) = \underbrace{-\log_{10}(\max(\delta,\ 10^{-9})) - 3}_{\text{gap signal } f(\delta)} + \underbrace{0.5 \cdot \left(\frac{s_2}{s_1} - 0.975\right)}_{\text{ratio signal } g(s_1,s_2)} - \underbrace{0.3 \cdot \mathbf{1}[\hat{s} = s_{j^*}]}_{\text{server match } h(\hat{s})} \tag{7}$$

where $\delta = s_1 - s_2$ is the score gap between the top-1 and top-2 results, s_1 and s_2 are the adjusted scores and $\hat{s}$ is any explicitly named server extracted from the query. The three terms capture complementary confidence signals. The first term, $f(\delta)$, applies a log transformation to the score gap, which spreads the gap values (typically in the range 10^{-4} to 10^{-2}) across the calibrated k-bucket boundaries. A gap of 0.001 yields f=0.0; a gap of 0.0001 yields f=1.0. The second term, $g(s_1, s_2)$, provides a secondary signal based on the score ratio, which is useful when absolute gap is small but the relative separation is informative. The third term, $h(\hat{s})$, provides a

structural confidence bonus of 0.3 when the user explicitly names a server and the top-ranked tool belongs to that server, since explicit name matches are rarely wrong.

Lower non-conformity scores indicate greater confidence. The score is mapped to three tiers via calibrated thresholds τ_1 and τ_3, as summarised in table 3.

Table 3: Confidence-based handoff tiers. Lower non-conformity score indicates higher confidence in the top-ranked tool.

Tier	**nc Condition**	**Handoff-k**	**Rationale**
High	$\text{nc} \leq \tau_1$	1	Clear top tool with strong structural and score-gap evidence; inject minimal context to preserve reasoning tokens.
Medium	$\tau_1 < \text{nc} \leq \tau_3$	3	Near-tie among top candidates; provide a small set so the LLM can disambiguate using conversation context.
Low	$\text{nc} > \tau_3$	5	Genuinely ambiguous; provide a larger candidate set to maximise coverage at the cost of additional tokens.

5.3 Calibration Procedure

The thresholds τ_1 and τ_3 are not hand-tuned but are calibrated from data. Given a held-out calibration set $\mathcal{D}$ of queries with known ground-truth tools, the algorithm computes the non-conformity-inspired score for each query, sorts the scores and searches for the threshold pair that achieves a target empirical coverage level (e.g., 95%) at each tier while minimising the average number of tools returned. The search proceeds over a quantile grid of the observed scores, ensuring that the thresholds are grounded in the actual score distribution rather than arbitrary constants. Because the deployed router combines this score with engineered retrieval features, these thresholds should be read as calibrated empirical handoff thresholds, not as a formal conformal prediction guarantee. The full procedure is given in algorithm 3.

Algorithm 3 Non-Conformity Threshold Calibration

Require: Calibration set $\mathcal{D}$, target coverage $1-\alpha$, average-k constraints $(k^{(1)}_{\max}, k^{(3)}_{\max})$
Ensure: Thresholds τ_1, τ_3

```
Compute NC = {nc(q) : q ∈ D}
Sort NC in ascending order
for each candidate τ1 in quantile grid of NC do
    Compute coverage@1 and average-k under τ1
    if coverage@1 ≥ 1−α and avg-k ≤ k^(1)_max then
        Accept τ1 and stop searching
Repeat the search analogously to find τ3 > τ1
return (τ1, τ3)
```

Calibration set details and runtime mode. For reproducibility, we make the calibration protocol explicit. The non-conformity thresholds are derived from `benchmarks.../single_tool.clean.json` (n=500 labelled routing cases). We use a deterministic split-conformal partition: records are sorted by case id and split 80/20 into a calibration set (n=400) and a held-out validation set (n=100); this protocol uses no random seed and no stratification. Leakage is prevented by construction because no case appears in both splits. The calibrated values stored in `benchmarks/results/conformal_calibration.json` are τ_1=−0.242075 and τ_3=−0.088911 (with τ_5=0.286132, τ_{null}=0.589435, target coverage 0.95). **Runtime mode for headline results:** the main functional-correctness tables in this paper are produced with `conformal_enabled=false` (gap-threshold handoff policy); non-conformity thresholds are reported as an optional calibrated policy and evaluated in the confidence analysis scripts.

6 Robustness and Safety Mechanisms

6.1 Null Routing and Abstention Guard

When no tool in the index genuinely matches the user's query, injecting low-quality candidates can have harmful downstream effects. The LLM, presented with tool schemas that bear superficial similarity to the intent, may fabricate a tool call with hallucinated arguments, leading to runtime errors or, worse, unintended side effects against a live API. VOTR addresses this with a two-stage abstention mechanism.

The first stage is the *abstention guard*. After re-ranking, the system computes a normalised query support score for the top-ranked candidate, defined as the weighted sum of field-level Sørensen–Dice overlaps across all five metadata fields, normalised by the total field weight. If this score falls below a threshold θ_q=0.18 and the server-level support (the maximum of server name and server summary overlaps) falls below θ_s=0.12, the confidence is downgraded to *low* and the handoff-k is floored at 5. This ensures that the downstream model receives a wide set of candidates along with a clear signal that the router is not confident, enabling the model to exercise its own judgement.

The second stage is the *null route*. If the query support falls below a stricter threshold θ_q^{null}=0.213 and the tool-level support (the maximum of tool name and tool description overlaps) does not exceed 0.4, the router returns an empty candidate list. This null route signal tells the agent that no suitable tool was found, preventing the model from attempting a tool call at all. The null route is only triggered when the query does not explicitly name a known server, since an explicit server match provides structural evidence that overrides the overlap-based support signal.

6.2 Overlap-Aware Disambiguation

A common characteristic of the MCP ecosystem is that multiple servers implement near-identical capabilities. For example, both a GitHub MCP server and a GitLab MCP server may expose a `search_repositories` operation with functionally equivalent parameter schemas. When the

user asks to "search for code repositories," both tools are legitimate matches, and the router's top-1 choice between them is essentially arbitrary.

To handle this, VOTR pre-computes a *capability signature* for each tool: a normalised, sorted concatenation of the tool's name tokens and parameter key tokens. Tools with identical capability signatures are grouped into overlap clusters. At routing time, if the top-ranked tool belongs to a cluster containing tools from two or more distinct servers whose scores fall within a window of δ_{ov}=0.0025 of the top score, VOTR flags the response as `overlap_ambiguous = true`, expands the handoff-k to cover all cluster members (up to a maximum of 3), lists the competing tool keys and server names in the response and downgrades the confidence tier by one level. This enables the downstream agent to present the user with a disambiguation prompt or to select based on user preference.

6.3 Abstention Regression Testing

To ensure that safety properties are preserved across code changes, VOTR maintains a curated regression suite of sixteen adversarial queries: eight policy-cleaned unsupported intents (queries for capabilities that no server in the index provides, e.g., "schedule an appointment with a dentist") and eight raw adversarial stress prompts (queries with misleading server name fragments or injection-style phrasing). The regression suite is run as part of the continuous evaluation pipeline and any code change that causes a previously null-routed query to return a non-empty result is flagged as a regression. The current pass rate is 100% on both suites.

7 Dynamic Registry and Hot Registration

7.1 The Registration Pipeline

The hot registration procedure, detailed in algorithm 4, enables new MCP servers to be added to the live index without restarting the service or rebuilding the existing embeddings. When a `RegisteredServer` object arrives at the `/register` endpoint, the system first validates that no server with the same name already exists in the index (to prevent accidental duplication). It then computes three sets of embeddings via the OpenAI embedding API: one vector for the server description, one for the server summary and one for each tool (formed by concatenating the tool name and description). These new vectors are concatenated with the existing embedding arrays via NumPy array operations, the server and tool index mapping arrays are extended, the server list is appended and the BM25 and SPLADE-lite indices are rebuilt over the extended document corpus. The updated index is persisted to disk and the engine's live references are atomically swapped.

7.2 Transport Discovery: stdio and SSE

The `/register/discover` endpoint automates the process of connecting to a live MCP server, extracting its tool catalogue and registering it. When the endpoint receives a command and argument list (e.g., `npx -y @modelcontextprotocol/server-filesystem C:\repo`), it spawns

Algorithm 4 Hot Server Registration

Require: New server $\mathbf{s}$ with tools $\{t_1, \ldots, t_m\}$, embedder E, current index $\mathcal{I}$
Ensure: Updated index $\mathcal{I}'$

Validate uniqueness of $\mathbf{s}$.name in $\mathcal{I}$
$\mathbf{e}_{\text{desc}} \leftarrow E(\mathbf{s}.\text{description})$
$\mathbf{e}_{\text{sum}} \leftarrow E(\mathbf{s}.\text{summary})$
for all $t_i \in \{t_1, \ldots, t_m\}$ **do**
    $\mathbf{e}_i \leftarrow E(t_i.\text{name} \parallel t_i.\text{description})$
Append $\mathbf{e}_{\text{desc}}$, $\mathbf{e}_{\text{sum}}$ to server embedding arrays
Append $\{\mathbf{e}_1, \ldots, \mathbf{e}_m\}$ to tool embedding array
Update server/tool index mapping arrays
Rebuild BM25 and SPLADE-lite indices over extended corpus
Persist updated shards to disk
return $\mathcal{I}' = \mathcal{I} \cup \mathbf{s}$

the target MCP server as a subprocess, writes an MCP `initialize` handshake over stdio, waits for the server's capability response, issues a `tools/list` request and collects the returned tool schemas. Each tool schema (containing a name, description and JSON-Schema input specification) is normalised into a `RegisteredTool` object and the collection is wrapped in a `RegisteredServer` with the user-provided name and description. The subprocess is then terminated and the standard registration pipeline takes over.

The SSE variant (`/register/discover/sse`) performs the same JSON-RPC calls over HTTP to a running endpoint rather than a subprocess, making it suitable for remote or containerised MCP servers. Both variants enforce a configurable timeout (2–120 seconds, default 20 seconds) to prevent hanging on unresponsive servers.

7.3 Zero-Downtime Index Updates

Registration is designed to be atomic from the query path's perspective. The new `ToolIndex` and `HybridRetriever` are fully constructed before any live state is modified. The engine's `index` and `hybrid` attributes are then replaced in a single Python reference assignment, which under CPython's Global Interpreter Lock (GIL) is atomic with respect to concurrent request threads. In-flight requests that acquired a reference to the old index before the swap complete normally using the old data; subsequent requests immediately use the new index. This design eliminates any downtime window during registration.

8 Efficiency Optimisations

8.1 Compressed Schema Injection

MCP-Zero injects full JSON-Schema blocks into the model's context for each retrieved tool. A typical tool schema includes the tool name, description, a `parameters` object with nested `properties`, `type` annotations, `required` arrays and optional `enum` or `default` values. MCP-Zero's own measurements report roughly 143 tokens per tool (Figure 2 in their paper); the exact count depends on corpus, schema verbosity and tokenizer.

VOTR replaces that payload with a *compact* representation emitted by `compress_tool_line` in the router implementation: a routing prefix, a parenthesised parameter list with coarse types, and a short natural-language description. The prefix is literally `[server: <name>]` where `<name>` is the MCP server identifier (any server, not GitHub-specific). Numeric parameters are summarised as `num` rather than `int` or `float`. The string spans two physical lines (signature, then an arrow and description), for example if the server name were `GitHub`:

```
[server: GitHub] search_repositories(query: str, page?: num,
perPage?: num)
→ Search for GitHub repositories matching a query.
```

This format retains the three pieces of information that the model needs to generate a correct function call: the server name (for routing), the function signature with parameter names and types (for argument construction) and a brief description (for intent verification).

Reproducible token counts. Token totals are not unique: they depend on the tokenizer shipped with the target model. We report counts from the `tiktoken` library using the `cl100k_base` encoding. The script `measure_schema_tokens.py` walks the on-disk tool index, formats each tool both as an MCP-Zero-style `<function>` JSON block and as VOTR's compressed line, and writes aggregates to `schema_token_stats.json`.

Table 4 quantifies the token savings.

Table 4: Token cost comparison: MCP-Zero-style JSON-Schema block vs. VOTR compressed schema (mean over all tools in the built index; `tiktoken`, `cl100k_base`).

Format	**Mean tokens/tool**	**At k=3**	**Reduction**
MCP-Zero paper (Figure 2)	≈143	≈429	—
MCP-Zero-style `<function>` (our index, n=2,806)	93.5	281	—
Compressed (VOTR, same index)	26.2	79	72%
End-to-end measured average across single_tool.large (500 queries):			
VOTR full-stack mean tokens	—	230.3	—

The first data row cites MCP-Zero's published average under their measurement setup. The next two rows are an apples-to-apples comparison on *this* paper's tool snapshot: the same 2,806 tools are formatted both ways and tokenised identically, so the 72% reduction isolates the compression effect from differences in tokenizer or catalogue. The lower mean (93.5 vs. 143) for the replicated MCP-Zero wrapper reflects shorter schemas in parts of our corpus relative to theirs.

At the measured average handoff-k of 2.9 tools, pure schema content is approximately $26.2 \times 2.9 \approx 76$ tokens under compression versus $93.5 \times 2.9 \approx 271$ tokens for MCP-Zero-style blocks; again about a 72% reduction. The total measured injection of 230.3 tokens in the last row includes compressed schemas *plus* other routing metadata returned with each `/route` response, so it is not equal to $26.2 \times k$ alone.

When compared against full-catalogue MCP-Zero-style injection on the same 2,806-tool index ($93.5 \times 2{,}806 \approx 262{,}487$ tokens), VOTR's measured 230.3 tokens per route corresponds to a 99.91% end-to-end token reduction.

8.2 Multi-Turn Session Memory

In a multi-turn conversation, the same tools are often relevant across consecutive turns. A user who asks to read a file in turn one and edit it in turn two likely needs filesystem tools in both turns, but re-injecting the same tool schemas wastes tokens since the model already has them in its conversation history.

VOTR maintains a thread-safe `SessionMemory` that associates each session identifier with a set of tool keys that have been returned in prior routing calls. The memory uses a TTL-based expiry mechanism (default 24 hours) with lazy garbage collection: on each write operation, sessions whose last activity exceeds the TTL are purged. When the `/route` endpoint is called with a `session_id`, the returned tools are recorded in the session's tool set. In the current implementation this state is recorded but not yet used to suppress re-injection of already-known tools. Future versions will use this information to provide cumulative token savings proportional to conversation length.

8.3 Index Backend Selection

Server counts in real-world MCP ecosystems range from tens to a few thousand. At this scale, the dense linear algebra used by VOTR's NumPy-based `ToolIndex` is adequate. Server routing normalises the query embedding and computes cosine similarity against all server description and summary vectors as two matrix–vector products, `_server_desc @ q.T` and `_server_sum @ q.T`, i.e. products of shape $(S, d) \times (d, 1)$ for S servers and embedding dimension d (3072 for `text-embedding-3-large` in our configuration). On the 309-server evaluation snapshot this is a few million multiply-adds per query; inexpensive on a single CPU core (we do not report an isolated micro-benchmark of the matvec in the repository). Tool scoring applies the same pattern to the subset of tool rows whose parent server lies among the top-N candidates (`tools_for_servers` followed by `score_tools_hierarchical` in the implementation).

Deployments beyond roughly 10^4 tools, strict multi-tenant isolation, or multi-replica serving may warrant an external vector database with approximate nearest-neighbour search (for example Qdrant [21] with HNSW). The public codebase includes a `QdrantStore` stub under `stores/qdrant_store.py` as a placeholder for such integration; the released routing path always loads `ToolIndex` from on-disk `.npy` shards and `meta.json`, and there is presently no configuration switch that routes retrieval through Qdrant.

9 Evaluation

9.1 Experimental Setup

All experiments are conducted on the slightly expanded upon MCP-Zero tool catalogue, which comprises 309 registered servers exposing a total of 2,806 tools. Embeddings are generated using OpenAI's `text-embedding-3-large` model, producing 3072-dimensional vectors. Figure 3 visualises the resulting embedding space: a PCA pre-reduction followed by t-SNE projection of all 2,806 tool vectors, coloured by ten k-means clusters. The plot reveals both the opportunity and the challenge of embedding-based retrieval. Well-separated peripheral clusters (e.g. the grey and pink regions) correspond to domain-specific server groups where dense retrieval is highly reliable. The dense central mixing zone, where tools from semantically adjacent servers interleave across cluster boundaries, is precisely where dense-only retrieval fails (93.8% top-1), BM25's exact-match signal provides a complementary correction, field-aware re-ranking resolves near-tie ambiguities and the confidence policy correctly assigns the low-confidence tier to harder queries.

The VOTR service runs as a single-process FastAPI application. Latency measurements include the full end-to-end routing time, encompassing the OpenAI embedding API round-trip (which dominates total latency), local retrieval and re-ranking computation and response serialisation. All results use the `full_stack` configuration unless otherwise noted, which enables all retrieval stages (dense, BM25, SPLADE-lite), field-aware re-ranking, adaptive-k and the confidence-based handoff policy. For system-level validation, we additionally use the orchestrator integration layer described in section 3.1 to verify that routed tool sets are directly consumable by a production-style tool-calling agent loop.

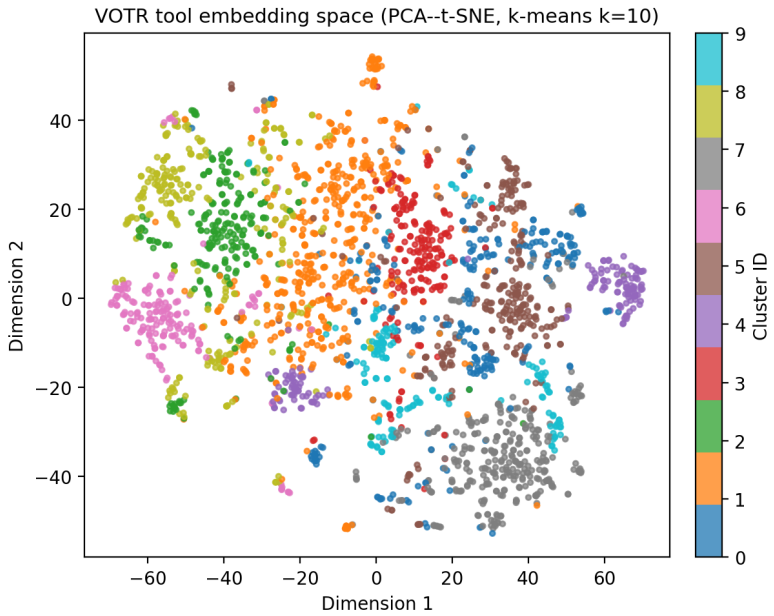

Figure 3: 2D projection of all 2,806 tool embeddings (PCA pre-reduction then t-SNE; coloured by ten k-means clusters). Well-separated peripheral clusters correspond to domain-specific server groups where dense retrieval achieves high precision.

The dense central mixing zone where tools from semantically adjacent servers share embedding neighbourhood motivates the hybrid BM25+SPLADE-lite retrieval, field-aware re-ranking and overlap-aware disambiguation described in sections 4 and 6.

9.2 Datasets and Benchmarks

Evaluation spans three task categories and three scale tiers, summarised in table 5. Column n counts **routing evaluations**: one per single-tool query, one per hop in multi-hop chains, and one per subtask in multi-tool turns.

Single-tool queries each target exactly one correct tool. The large suite contains 500 queries drawn from across the full 309-server catalogue. The medium suite contains 250 queries over a 50-server subset (cycled from a curated 88-query pool so that n matches other tiers). Three small subsets target specific server ecosystems: Bloomberg, GitHub and Telegram, each with 100 queries (cycled from the original labelled pools for those servers).

Multi-hop queries consist of chains of dependent routing requests where each hop's tool is a prerequisite for the next. Tiered suites use **five hops per case** at small (20 cases), medium (50 cases) and large (100 cases), giving $n \in \{100, 250, 500\}$ hop evaluations aligned with single-tool. Chains are constructed from consecutive labelled single-tool rows drawn from the appropriate catalogue pool so that hop-level difficulty is comparable to the single-tool suites at the same scale. Separately, extended stress tests of 10, 20, 25 and 50 hops (including adversarial variants) use the standalone multi-hop evaluator in table 19.

Multi-tool queries require identifying multiple distinct correct tools from a single user turn, with the same n and five-subtasks-per-case structure as multi-hop for fair cross-suite comparison.

Additionally, VOTR is evaluated on LiveMCPBench [22], a benchmark derived from real MCP agent traces in the live ecosystem, providing an out-of-distribution stress test.

Table 5: Evaluation datasets. n is the number of routing evaluations (one per query, hop, or subtask). Multi-hop and multi-tool tiers use five steps per case (20 / 50 / 100 cases).

Task Type	**Scale**	n	**Notes**
single_tool	small (Bloomberg)	100	1 server, cycled pool
single_tool	small (GitHub)	100	1 server, cycled pool
single_tool	small (Telegram)	100	1 server, cycled pool
single_tool	medium (50 servers)	250	50-server subset
single_tool	large (309 servers)	500	Full catalogue
multi_hop	small	100	20×5 hops, 3-server pool
multi_hop	medium	250	50×5 hops, 50-server pool
multi_hop	large	500	100×5 hops, full catalogue
multi_tool	small	100	20×5 subtasks
multi_tool	medium	250	50×5 subtasks
multi_tool	large	500	100×5 subtasks

9.3 Benchmark Provenance and Independence

Because retrieval quality can be overstated when benchmark construction is not independent from index construction, we make the provenance assumptions explicit. The evaluated VOTR index is built from the MCP-Zero-derived catalogue snapshot (308 servers, 2,797 tools) expanded with an additional Bloomberg BLPAPI MCP server, and the primary `single_tool.clean` benchmark is also defined over that same closed catalogue, with one labelled `expected_tool_key` per case. Accordingly, the reported 96.8% Top-1 should be interpreted as *closed-catalogue routing accuracy*, not unseen-catalogue generalisation excluding the added Bloomberg BLPAPI MCP server.

The scaled suites used in this paper are generated deterministically from labelled pools. As implemented in `build_scaled_suites.py`, `single_tool.medium_250` is formed by cycling a curated medium pool; `multi_hop.*` suites are constructed by chaining consecutive single-tool rows into fixed five-hop cases; and `multi_tool.*` suites analogously compose five labelled subtasks per case. A small subset of medium and small test suites were human-authored and were derived from a mix of catalogue entries and unseen MCP embeddings. This process preserves label quality and comparability across scales, but it does not provide strict statistical independence between the base pool and derived multi-hop/multi-tool suites.

We therefore include three safeguards. First, we report strict metrics and equivalence-aware metrics separately, so alias resolution does not inflate the headline strict scores. Second, we evaluate negative-control unsupported and adversarial query suites to test abstention behaviour rather than only in-catalogue matching. Third, we foreground LiveMCPBench as an external stress test whose task traces are separate from the functional-correctness suite construction. For LiveMCPBench specifically, our evaluation is prepared from the same 95-task annotation source and derived 268-step reconstruction file used by the benchmark tooling in this repository. The LiveMCPBench index itself is built separately from the benchmark's server/tool catalog metadata via `scripts/build_livemcpbench_embedding_json.py`; that build path embeds only server descriptions, summaries, tool names, descriptions and parameter metadata, and does not consume task questions or step-query text during index construction. This gives an external validation signal for interface robustness, while future revisions should further strengthen independence by evaluating on fully held-out human-authored query sets that are not derived from catalogue-labelled pools.

Reproducibility. Unless otherwise stated, the main VOTR results use the MCP-Zero-derived catalogue snapshot expanded with the Bloomberg BLPAPI MCP server (309 servers and 2,806 tools after indexing) and OpenAI `text-embedding-3-large` with 3072-dimensional vectors. Alternative embedding-backbone experiments rebuild the same metadata into separate index directories using `BAAI/bge-large-en-v1.5` (1024 dimensions), `google/embeddinggemma-300m`, `google/gemini-embedding-2-preview` and `qwen/qwen3-embedding-8b`. Functional correctness results are computed by issuing router `/route` calls against the suite JSON files under `benchmarks/functional_correctness`, with equivalence-aware metrics enabled via `equivalence_map.json`. LiveMCPBench results use the prepared artefacts under `evaluation/results/livemcpbench` and are reported only for the default embedding path because those rows reflect the paper's

orchestrator-mediated input contract. Latency measurements are per-route wall-clock timings on the local development host; API-backed models include provider round-trip latency, whereas local BGE runs include local SentenceTransformer inference time.

9.4 Evaluation Metrics

Top-k Accuracy measures the fraction of queries for which the ground-truth tool appears within the top-k results returned by the router. This is the primary retrieval quality metric, reported at $k \in \{1, 3, 5\}$.

Handoff@k measures accuracy using only the `recommended_handoff_k` candidates determined by the confidence policy, rather than a fixed k. This metric captures the effectiveness of the confidence-gated selection: a system that returns fewer tools on average while maintaining accuracy is preferable.

Chain Success@k is the fraction of multi-hop cases where every hop in the chain achieves Hit@k. This is an all-or-nothing metric: a single missed hop causes the entire chain to fail. The analogous metric for multi-tool tasks is *All-targets Success@k*, requiring every subtask to hit within top-k.

Equivalence-aware Top-1 recomputes hits after collapsing known duplicate or alias tool keys using a fixed equivalence map shipped with the benchmark; it isolates naming inconsistency from genuine retrieval errors.

mAP@5 (mean Average Precision at rank 5) and *nDCG@5* (normalised Discounted Cumulative Gain at rank 5) are standard information-retrieval metrics used for the LiveMCPBench evaluation.

Latency P50/P95 is the end-to-end routing latency in milliseconds, measured at the 50th and 95th percentiles across all queries in each suite.

Mean Estimated Tokens is the average number of tokens injected into the model's context per routing decision, estimated from the compressed schema lengths.

9.5 Functional Correctness Results

Table 6 presents the full-stack functional correctness results across all suite types and scales. **Hop-level** metrics (Top-1/3/5, Handoff@k) for multi-hop and multi-tool are computed over every hop or subtask, analogously to single-tool queries. At large scale, hop-level performance remains close to the single-tool suite (96.8% Top-1) because both draw from the same 500 labelled routing steps. **Compound** metrics (Chain / All-targets Success@k) require every step in a five-step case to succeed; these are stricter and surface the multiplicative effect of rare per-hop errors (89% at k=1, 94% at k=3, 97% at k=5 on the large catalogue). Equivalence-aware hop-level Top-1 reaches 97.4% and compound Success@1 reaches 92% (fig. 4).

Table 6: Full-stack functional correctness. Top-k and Handoff columns are hop/subtask-level for multi suites. Chn@k = Chain (multi-hop) or All-targets (multi-tool) Success@k. Eq. T1 = equivalence-aware hop Top-1; Chn@1^{eq} uses equivalence-aware hits per hop.

Task	**Scale**	**T1**	**T3**	**T5**	**Hd@k**	**Chn@1**	**Chn@3**	**Chn@5**
single_tool (equivalence and compound columns omitted)								
	s. Bloomberg	1.000	1.000	1.000	1.000	—	—	—
	s. GitHub	1.000	1.000	1.000	1.000	—	—	—
	s. Telegram	1.000	1.000	1.000	1.000	—	—	—
	medium	1.000	1.000	1.000	1.000	—	—	—
	large	0.968	0.986	0.992	0.984	—	—	—
multi_hop: Eq. T1 small/med/large = 1.000 / 0.992 / 0.974; Chn@1^{eq} = 1.00 / 1.00 / 0.92								
	small	1.000	1.000	1.000	1.000	1.000	1.000	1.000
	medium	1.000	1.000	1.000	1.000	1.000	1.000	1.000
	large	0.968	0.986	0.992	0.984	0.900	0.950	0.970
multi_tool: hop-level columns identical to *multi_hop* at each scale.								
	small	1.000	1.000	1.000	1.000	1.000	1.000	1.000
	medium	1.000	1.000	1.000	1.000	1.000	1.000	1.000
	large	0.968	0.986	0.992	0.984	0.900	0.950	0.970

Single-tool accuracy on the three small subsets (100 queries each) remains 100% across all metrics. The medium suite (250 queries, 50 servers) also reaches 100% Top-1/3/5 and Handoff@k. The large suite (500 queries) achieves 96.8% Top-1, 98.6% Top-3, 99.2% Top-5 and 98.4% Handoff@k, with equivalence-aware Top-1 at 97.4%. On `multi_hop.large`, the full-stack router reports 96.8% Top-1, 98.6% Top-3, 99.2% Top-5, 98.4% Handoff@k, and chain Success@k of 90% / 95% / 97% at k=1/3/5. The gap between Top-1 and Handoff@k on the large suite (96.8% vs. 98.4%) demonstrates the value of the confidence policy: when the top-1 result is incorrect, the policy often assigns lower confidence and returns additional candidates that include the correct tool.

To quantify sampling variability, table 7 reports non-parametric bootstrap confidence intervals for the headline metrics. We resample routing items for hop/subtask-level metrics and resample cases for chain/all-target metrics using 10,000 bootstrap replicates with a fixed seed. The large single-tool, multi-hop and multi-tool suites have narrow item-level intervals, while the 16-case robustness suite has a wider interval, as expected for a small stress-test set.

Table 7: Bootstrap 95% confidence intervals for headline metrics. Intervals are computed from existing report JSONs using 10,000 non-parametric bootstrap replicates.

Suite	**Metric**	n	**Estimate [95% CI]**
single_tool.large	Top-1	500	0.968 [0.952, 0.982]
	Handoff@k	500	0.984 [0.972, 0.994]
multi_hop.large	Top-1	500	0.968 [0.952, 0.982]
	Handoff@k	500	0.984 [0.972, 0.994]
	Chain@1	100	0.900 [0.840, 0.950]
multi_tool.large	Top-1	500	0.968 [0.952, 0.982]
	Handoff@k	500	0.984 [0.972, 0.994]
	All-target@1	100	0.900 [0.840, 0.950]
robustness_safety	Top-1	16	0.625 [0.375, 0.875]
	Handoff@k	16	0.625 [0.375, 0.875]

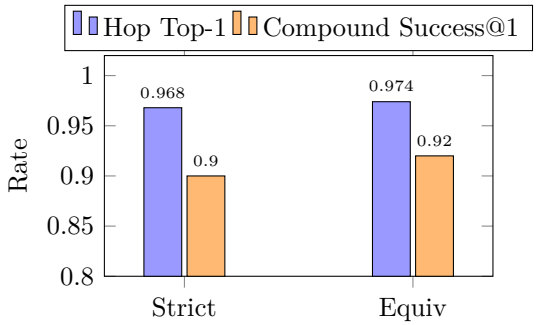

Figure 4: Strict vs. equivalence-aware metrics on the large catalogue (n=500 hop evaluations; 100 five-step multi cases). Equivalence recovers four hop-level strict misses; compound Success@1 gains three percentage points.

Table 8 lists every strict Top-1 miss on `single_tool.large`: 16 in total (484/500 correct). Most failure mass lies in the *low* confidence tier: about 46% of steps are labelled low-confidence, and they account for 9 of the 16 misses (≈3.9% within-tier error vs. 2.8% for high and 2.2% for medium).

Table 8: Strict Top-1 miss accounting by confidence tier on `single_tool.large` (n=500 routing steps, full-stack).

Tier	**Misses**	**Steps in tier**	**Within-tier err. rate**	**Share of all misses**
High	5	178	2.8%	31%
Medium	2	93	2.2%	13%
Low	9	229	3.9%	56%
Total	**16**	**500**	3.2%	100%

Equivalence-aware labelling recovers **three** of the strict misses (alias groups), raising hop-level Top-1 from 96.8% to 97.4% (+0.6 percentage points overall, i.e. three misses out of sixteen). There are **no** cases where strict Top-1 succeeds but equivalence fails. On large multi-hop/multi-tool suites, the same rescues lift compound Success@1 from 90% to 92% (two additional fully successful five-step cases).

9.6 Embedding Backbone Comparison (OpenAI vs. Local vs. Gemini vs. Qwen)

To evaluate embedding-backbone sensitivity while keeping the retrieval stack fixed, we ran isolated pipelines for four alternatives to the default OpenAI embedding path: `BAAI/bge-large-en-v1.5` (local), `google/embeddinggemma-300m` (local), `qwen/qwen3-embedding-8b` (local), and Gemini Embedding 2 Preview (`google/gemini-embedding-2-preview`, Google API). Each pipeline uses the same VOTR retrieval stages and hyperparameters, differing only in embedding model and pre-built index artifacts.

Table 9 reports OpenAI, Gemini Embedding 2, BGE, Qwen and Gemma on matched suites under a fixed router configuration. On `multi_hop.large`, OpenAI and Gemini are tied at Top-1 (0.968); Gemini is marginally higher at Top-3/Top-5 (0.990/0.996 vs. 0.986/0.992), while OpenAI is marginally higher at Handoff@k (0.984 vs. 0.980). For VOTR, this distinction is substantive: Top-3/Top-5 quantify candidate-set recall, whereas Top-1, Handoff@k, and compound Chain@All-target Success quantify end-to-end orchestration reliability under first-choice execution. On the largest multi-hop and multi-tool suites, OpenAI remains the slightly stronger orchestration anchor, matching Gemini on hop-level Top-1 while maintaining a small advantage on strict Chain@All-target Success@1; Gemini correspondingly shows a small advantage on recall-oriented metrics such as Top-3/Top-5 and Success@5. BGE remains close to the two API models on large suites, while Qwen tracks below BGE/Gemini on medium/large compositional suites and Gemma exhibits the largest degradation on harder settings. On `multi_hop.medium_250`, Qwen reports 0.940 Top-1 and 0.700 Chain@1, while OpenAI, Gemini and BGE remain saturated at 1.000 on the same suite.

Table 9: Embedding-backbone comparison on matched suites across OpenAI, Gemini Embedding 2, BGE, Qwen and Gemma under a fixed retrieval stack.

Suite	Model	Top-1	Top-3	Top-5	Handoff@k	Chain@1
ambiguity_collision	OpenAI	0.889	0.944	1.000	0.889	—
	BGE	0.778	1.000	1.000	0.889	—
	Qwen	0.889	1.000	1.000	1.000	—
	Gemma	0.889	1.000	1.000	0.944	—
	Gemini2	0.889	1.000	1.000	1.000	—
multi_hop.large	OpenAI	0.968	0.986	0.992	0.984	0.900
	BGE	0.962	0.976	0.984	0.972	0.860
	Qwen	0.948	0.978	0.982	0.962	0.810
	Gemma	0.878	0.928	0.952	0.894	0.620
	Gemini2	0.968	0.990	0.996	0.980	0.890
multi_hop.medium_250	OpenAI	1.000	1.000	1.000	1.000	1.000
	BGE	1.000	1.000	1.000	1.000	1.000
	Qwen	0.940	1.000	1.000	0.964	0.700
	Gemma	0.840	0.916	0.928	0.864	0.460
	Gemini2	1.000	1.000	1.000	1.000	1.000
multi_hop.small_100	OpenAI	1.000	1.000	1.000	1.000	1.000
	BGE	1.000	1.000	1.000	1.000	1.000
	Qwen	1.000	1.000	1.000	1.000	1.000
	Gemma	1.000	1.000	1.000	1.000	1.000
	Gemini2	1.000	1.000	1.000	1.000	1.000

For reporting completeness and direct cross-model comparability, Tables 10 and 11 provide per-suite metric tuples for all evaluated backbones.

Table 10: Detailed hop/subtask metrics by suite and model. Each cell reports Top-1/Top-3/Top-5/Handoff@k/Eq-Top-1.

Suite	Model	Top-1	Top-3	Top-5	Hd@k	Eq-T1
`ambiguity_collision.priority`	OpenAI	0.8889	0.9444	1.0000	0.8889	0.9444
	BGE	0.7778	1.0000	1.0000	0.8889	0.8889
	Qwen	0.8889	1.0000	1.0000	1.0000	0.9444
	Gemma	0.8889	1.0000	1.0000	0.9444	0.9444
	Gemini2	0.8889	1.0000	1.0000	1.0000	0.9444
`multi_hop.large.cross_app`	OpenAI	0.9680	0.9860	0.9920	0.9840	0.9740
	BGE	0.9620	0.9760	0.9840	0.9720	0.9660
	Qwen	0.9480	0.9780	0.9820	0.9620	0.9480
	Gemma	0.8780	0.9280	0.9520	0.8940	0.8800

Continued on next page

Table 10: Detailed hop/subtask metrics by suite and model (continued).

Suite	Model	Top-1	Top-3	Top-5	Hd@k	Eq-T1
	Gemini2	0.9680	0.9900	0.9960	0.9800	0.9720
multi_hop.medium_250.cross_app	OpenAI	1.0000	1.0000	1.0000	1.0000	1.0000
	BGE	1.0000	1.0000	1.0000	1.0000	1.0000
	Qwen	0.9400	1.0000	1.0000	0.9640	0.9400
	Gemma	0.8400	0.9160	0.9280	0.8640	0.8400
	Gemini2	1.0000	1.0000	1.0000	1.0000	1.0000
multi_hop.small_100.cross_app	OpenAI	1.0000	1.0000	1.0000	1.0000	1.0000
	BGE	1.0000	1.0000	1.0000	1.0000	1.0000
	Qwen	1.0000	1.0000	1.0000	1.0000	1.0000
	Gemma	1.0000	1.0000	1.0000	1.0000	1.0000
	Gemini2	1.0000	1.0000	1.0000	1.0000	1.0000
multi_tool.large.single_turn	OpenAI	0.9680	0.9860	0.9920	0.9840	0.9740
	BGE	0.9620	0.9760	0.9840	0.9720	0.9660
	Qwen	0.9480	0.9780	0.9820	0.9620	0.9480
	Gemma	0.8780	0.9280	0.9520	0.8940	0.8800
	Gemini2	0.9680	0.9900	0.9960	0.9800	0.9720
multi_tool.medium_250.single_turn	OpenAI	1.0000	1.0000	1.0000	1.0000	1.0000
	BGE	1.0000	1.0000	1.0000	1.0000	1.0000
	Qwen	0.9400	1.0000	1.0000	0.9640	0.9400
	Gemma	0.8400	0.9160	0.9280	0.8640	0.8400
	Gemini2	1.0000	1.0000	1.0000	1.0000	1.0000
multi_tool.small_100.single_turn	OpenAI	1.0000	1.0000	1.0000	1.0000	1.0000
	BGE	1.0000	1.0000	1.0000	1.0000	1.0000
	Qwen	1.0000	1.0000	1.0000	1.0000	1.0000
	Gemma	1.0000	1.0000	1.0000	1.0000	1.0000
	Gemini2	1.0000	1.0000	1.0000	1.0000	1.0000
robustness_safety.priority	OpenAI	0.6250	0.6875	0.6875	0.6250	0.6250
	BGE	0.6250	0.6875	0.6875	0.6250	0.6250
	Qwen	0.5000	0.6875	0.6875	0.6250	0.5000
	Gemma	0.4375	0.6250	0.6250	0.5625	0.4375
	Gemini2	0.5000	0.6875	0.6875	0.6250	0.5000
single_tool.bloomberg.clean	OpenAI	1.0000	1.0000	1.0000	1.0000	1.0000
	BGE	1.0000	1.0000	1.0000	1.0000	1.0000
	Qwen	1.0000	1.0000	1.0000	1.0000	1.0000
	Gemma	1.0000	1.0000	1.0000	1.0000	1.0000
	Gemini2	1.0000	1.0000	1.0000	1.0000	1.0000
single_tool.clean	OpenAI	0.9680	0.9860	0.9920	0.9840	0.9740
	BGE	0.9620	0.9760	0.9840	0.9720	0.9660
	Qwen	0.9480	0.9780	0.9820	0.9620	0.9480
	Gemma	0.8780	0.9280	0.9520	0.8940	0.8800
	Gemini2	0.9680	0.9900	0.9960	0.9800	0.9720

Table 11: Detailed chain/all-target rates on compositional suites by model. Each cell reports strict@1/3/5 | eq@1/3/5.

Suite	Model	Strict @ 1/3/5	Eq @ 1/3/5
multi_hop.large	OpenAI	0.9000/0.9500/0.9700	0.9200/0.9600/0.9700
	BGE	0.8600/0.9200/0.9500	0.8700/0.9200/0.9500
	Qwen	0.8100/0.9100/0.9200	0.8100/0.9100/0.9200
	Gemma	0.6200/0.7200/0.7900	0.6200/0.7200/0.7900
	Gemini2	0.8900/0.9500/0.9800	0.9100/0.9500/0.9800
multi_hop.medium_250	OpenAI	1.0000/1.0000/1.0000	1.0000/1.0000/1.0000
	BGE	1.0000/1.0000/1.0000	1.0000/1.0000/1.0000
	Qwen	0.7000/1.0000/1.0000	0.7000/1.0000/1.0000
	Gemma	0.4600/0.6000/0.6600	0.4600/0.6000/0.6600
	Gemini2	1.0000/1.0000/1.0000	1.0000/1.0000/1.0000
multi_hop.small_100	OpenAI	1.0000/1.0000/1.0000	1.0000/1.0000/1.0000
	BGE	1.0000/1.0000/1.0000	1.0000/1.0000/1.0000
	Qwen	1.0000/1.0000/1.0000	1.0000/1.0000/1.0000
	Gemma	1.0000/1.0000/1.0000	1.0000/1.0000/1.0000
	Gemini2	1.0000/1.0000/1.0000	1.0000/1.0000/1.0000
multi_tool.large	OpenAI	0.9000/0.9500/0.9700	0.9200/0.9600/0.9700
	BGE	0.8600/0.9200/0.9500	0.8700/0.9200/0.9500
	Qwen	0.8100/0.9100/0.9200	0.8100/0.9100/0.9200
	Gemma	0.6200/0.7200/0.7900	0.6200/0.7200/0.7900
	Gemini2	0.8900/0.9500/0.9800	0.9100/0.9500/0.9800
multi_tool.medium_250	OpenAI	1.0000/1.0000/1.0000	1.0000/1.0000/1.0000
	BGE	1.0000/1.0000/1.0000	1.0000/1.0000/1.0000
	Qwen	0.7000/1.0000/1.0000	0.7000/1.0000/1.0000
	Gemma	0.4600/0.6000/0.6600	0.4600/0.6000/0.6600
	Gemini2	1.0000/1.0000/1.0000	1.0000/1.0000/1.0000
multi_tool.small_100	OpenAI	1.0000/1.0000/1.0000	1.0000/1.0000/1.0000
	BGE	1.0000/1.0000/1.0000	1.0000/1.0000/1.0000
	Qwen	1.0000/1.0000/1.0000	1.0000/1.0000/1.0000
	Gemma	1.0000/1.0000/1.0000	1.0000/1.0000/1.0000
	Gemini2	1.0000/1.0000/1.0000	1.0000/1.0000/1.0000

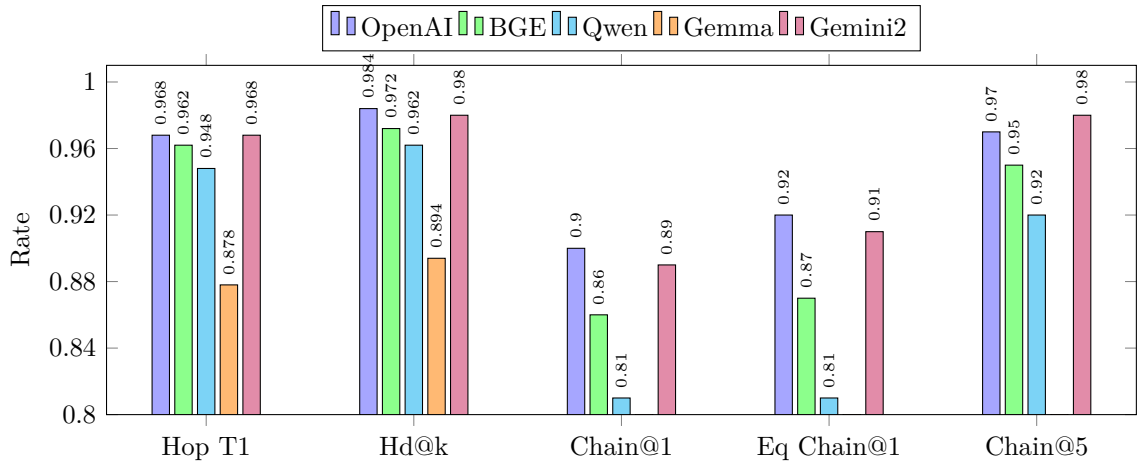

Figure 5: Large-suite orchestration reliability by embedding backbone on `multi_hop.large`. Gemini2 slightly improves recall-oriented metrics (Top-3/Top-5, reflected in Chain@5), OpenAI remains the strongest first-choice baseline, and Qwen sits between BGE and Gemma on strict orchestration rates.

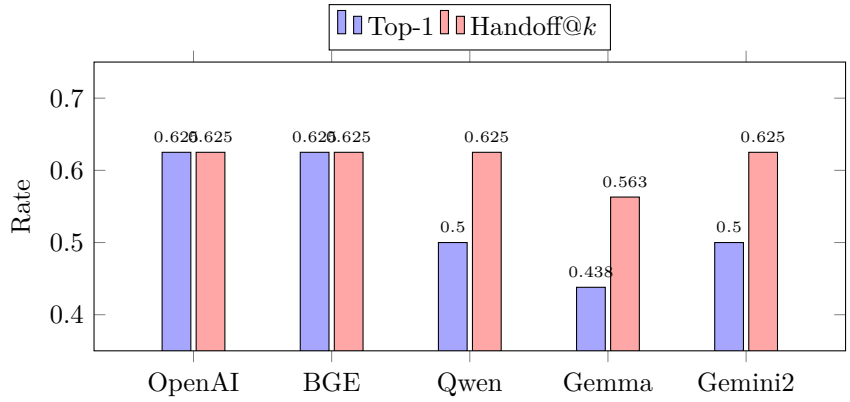

Figure 6: Robustness/safety priority suite by embedding backbone. Gemini2 and Qwen match OpenAI/BGE on Handoff@k but trail on Top-1 robustness score, which motivates retaining OpenAI as the primary high-performance baseline for safety-sensitive orchestration.

We continue to treat OpenAI `text-embedding-3-large` as the primary baseline for five methodological reasons. First, it is the canonical configuration used throughout this paper's main evaluation, calibration, ablation and latency sections, so keeping it as anchor preserves comparability with all earlier tables and claims. Second, it provides the most reliable first-choice orchestration signal among the proprietary models in the hardest settings: on `multi_hop.large` and `multi_tool.large`, OpenAI matches Gemini2 at hop/subtask Top-1 but is slightly higher on strict Chain@All-target Success@1 (0.900 vs. 0.890) and Handoff@k (0.984 vs. 0.980). Third, it is stronger on robustness/safety Top-1 (0.625 vs. 0.500), and this suite is the closest benchmark analogue to noisy user intent, ambiguous requests and safety-sensitive routing failures. For a scalable multi-agent router, confident wrong first choices are more damaging than missing a lower-ranked recall candidate. Fourth, Gemini is evaluated here as `google/gemini-embedding-2-preview`; despite its strong results, using a preview model as the default baseline is less conservative for

reproducibility and deployment stability because provider behaviour, availability and naming can change before general availability. Fifth, baseline selection for a scalable system is also an operational question: in our runs, Gemini required more careful throttling/checkpointing to handle provider-side limits and tiered quota behaviour, whereas the OpenAI baseline path was more ergonomic to execute repeatedly at full-suite scale. This does not reduce Gemini's retrieval quality; it affects accessibility and repeatability for other teams reproducing the paper under varied account tiers. Sixth, using one fixed high-capacity baseline reduces evaluation drift when introducing additional backbones. Gemini Embedding 2 Preview is therefore positioned as a strong comparative API model rather than the primary baseline: it matches OpenAI on large-suite Top-1 and improves several Top-3/Top-5 and Success@5 metrics, strengthening external validity of VOTR beyond a single provider. Among local models, `bge-large-en-v1.5` is the strongest in our runs and is the appropriate reproducible/scalable local baseline; it is notably closer to OpenAI/Gemini than Gemma on larger suites. `qwen3-embedding-8b` is an additional open-weight backbone run locally with the same indexing and routing workflow as BGE and Gemma.

9.7 Low-Cardinality Performance

Although the primary objective of VOTR is scalable retrieval over large and heterogeneous MCP ecosystems, the same architecture is also intended to serve early-stage and enterprise deployments where only one or two MCP servers are connected. This setting is operationally important because many teams begin with a narrowly scoped internal tool stack before scaling to a broader ecosystem. In such deployments, retrieval quality must remain near-perfect so that operators can trust the router before introducing additional servers.

The one-server benchmarks provide strong evidence for this deployment mode. For a newly registered, dynamically embedded MCP server, in single `_tool.small_bloomberg` /($n = 100$), VOTR achieves 1.000 for Top-1, Top-3, Top-5 and Handoff@k. The same ceiling-level pattern holds on `single_tool.small_github` and `single_tool.small_telegram` (each n=100), each with 1.000 across all retrieval metrics. These results show that VOTR's scalability-oriented hybrid stack does not sacrifice precision in low-cardinality environments; instead, it preserves top-tier accuracy while retaining a migration path to larger server catalogues.

9.8 Ablation Study

To understand the contribution of each retrieval component and the sensitivity of those components to the embedding backbone, we evaluate six profiles on the scaled `multi_hop.large.cross_app` suite (100 chains, 500 hop-level routing decisions). The profiles are `dense_only` (dense embedding retrieval without sparse stages), `bm25_only` (lexical retrieval without dense or SPLADE-lite), `dense_bm25` (the two primary signals without SPLADE-lite), `full_stack` (dense, BM25, SPLADE-lite, field-aware re-ranking and adaptive handoff), `no_handoff_policy` (full retrieval with a fixed handoff width), and `no_session_memory` (full retrieval with session tracking disabled). Table 12 reports the default OpenAI component ablation, while table 13 and fig. 7 compare the same profiles across all embedding backbones.

The default OpenAI ablation shows that lexical retrieval is an unusually strong baseline for this benchmark: BM25-only reaches 0.990 hop-level Top-1 and 0.998 Handoff@k. This reflects the benchmark's catalogue-derived tool descriptions, where many intents preserve exact server or capability terms. Dense-only retrieval is lower at 0.938 Top-1 but captures semantic matches that are useful outside exact-name matching. The production `full_stack` profile uses the primary functional-correctness report as the canonical source of truth, reaching 0.968 Top-1, 0.992 Top-5 and 0.984 Handoff@k. The remaining component profiles are diagnostic ablations run on the same 500-hop suite; they show how lexical, dense, sparse and handoff components affect routing under controlled profile changes. Disabling the handoff policy leaves strict ranking behaviour essentially unchanged but increases the handoff width to the fixed maximum, confirming that the policy primarily controls context budget rather than retrieval ranking.

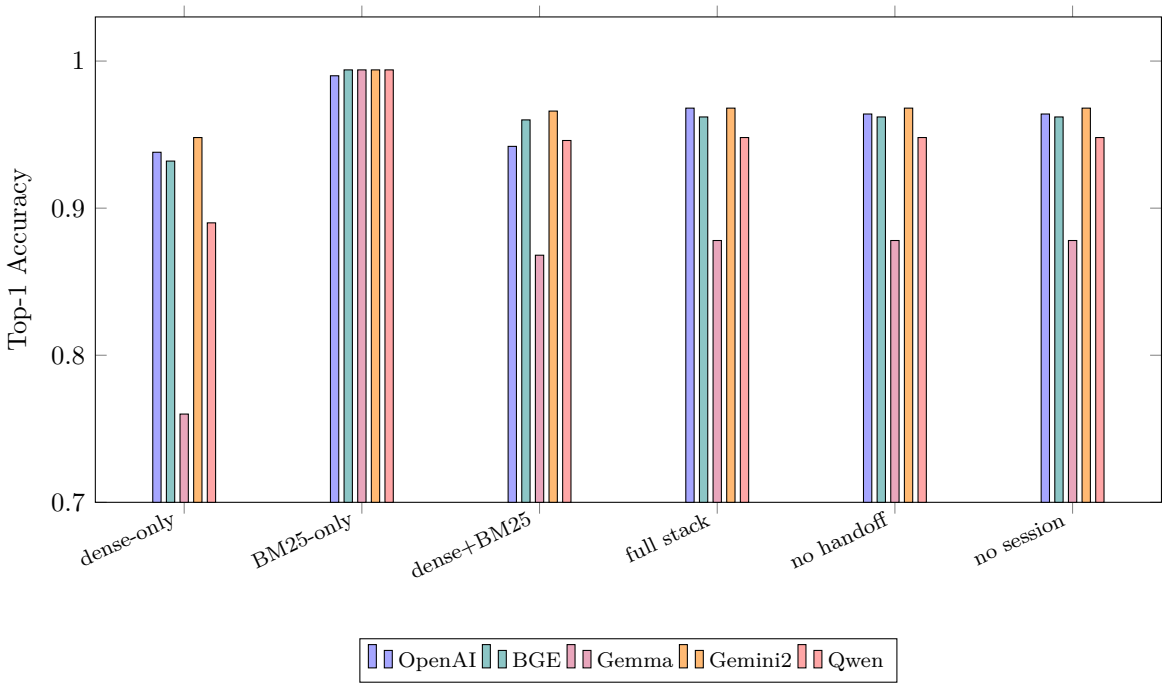

Figure 7: Hop-level Top-1 accuracy by retrieval profile and embedding backbone on the scaled `multi_hop.large.cross_app` suite. BM25-only is a strong lexical baseline across all backbones; the dense and hybrid profiles reveal the quality differences introduced by the embedding model.

Table 12: Default OpenAI component ablation on the scaled `multi_hop.large.cross_app` suite (500 hop-level decisions). The production `full_stack` row uses the primary functional-correctness report as the canonical source of truth; other rows are diagnostic profile ablations. Chain@1 is the fraction of 5-hop chains in which every hop is correct at rank 1.

Profile	**Top-1**	**Top-3**	**Top-5**	**Handoff@k**	**Avg. k**	**Chain@1**
dense_only	0.938	0.950	0.952	0.952	4.984	0.790
bm25_only	**0.990**	**0.998**	**0.998**	**0.998**	4.996	**0.960**
dense_bm25	0.942	0.974	0.986	0.980	4.036	0.810
full_stack	0.968	0.986	0.992	0.984	3.200	0.900
no_handoff_policy	0.964	0.980	0.990	0.992	8.000	0.870
no_session_memory	0.964	0.980	0.990	0.982	2.906	0.870

Table 13 extends the same profile sweep to BGE, Gemma, Gemini2 and Qwen. The BM25-only row is nearly identical across backbones because it does not use embedding vectors for ranking. The dense-only row is therefore the clearest isolation of embedding quality: Gemini2 is highest (0.948), followed by OpenAI (0.938), BGE (0.932), Qwen (0.890) and Gemma (0.760). Adding BM25 and SPLADE-lite narrows these gaps for BGE and Qwen, while Gemma remains materially lower on both hop-level and chain-level metrics. Session memory has no measurable effect on this suite because the benchmark hops already contain explicit server and tool intents; its value is expected to be larger in underspecified conversational workloads.

Table 13: Hop-level Top-1 ablation by embedding backbone on `multi_hop.large.cross_app`. Rows share the same 500-hop suite and the same router stack, changing only the embedding/index backbone and ablation profile.

Backbone	**Dense**	**BM25**	**Dense+BM25**	**Full**	**No Handoff**	**No Session**
OpenAI text-embedding-3-large	0.938	0.990	0.942	0.968	0.964	0.964
BGE large-en-v1.5	0.932	0.994	0.960	0.962	0.962	0.962
Gemma embedding-300m	0.760	0.994	0.868	0.878	0.878	0.878
Gemini Embedding 2 Preview	**0.948**	0.994	**0.966**	**0.968**	**0.968**	**0.968**
Qwen3 Embedding 8B	0.890	0.994	0.946	0.948	0.948	0.948

Table 14: Full-stack backbone comparison on `multi_hop.large.cross_app`. The OpenAI row uses the primary functional-correctness report as the canonical baseline; alternative-backbone rows use the matched backbone sweep. Handoff@k uses each model's adaptive handoff policy; Chain@1 and Chain@5 measure whether all five hops in a chain are recovered at the corresponding rank cutoff.

Backbone	**Top-1**	**Top-3**	**Top-5**	**Handoff@k**	**Chain@1**	**Chain@5**
OpenAI text-embedding-3-large	0.968	0.986	0.992	0.984	0.900	0.970
BGE large-en-v1.5	0.962	0.976	0.984	0.972	0.860	0.950
Gemma embedding-300m	0.878	0.928	0.952	0.894	0.620	0.790
Gemini Embedding 2 Preview	**0.968**	**0.990**	**0.996**	0.980	**0.890**	**0.980**
Qwen3 Embedding 8B	0.948	0.978	0.982	0.962	0.810	0.920

9.9 Confidence Calibration Results

Table 15 and fig. 8 report the accuracy of each confidence tier on the single-tool large suite. Of the 500 queries, 178 (35.6%) are assigned high confidence, 93 (18.6%) medium and 229 (45.8%) low. The high-confidence tier achieves 97.2% top-1 accuracy with handoff-k=1, meaning that over one-third of all queries are resolved with a single injected tool. The medium tier achieves 97.8% top-1 and perfect Handoff@k. The low-confidence tier achieves 96.1% top-1 but 98.7% Handoff@k, which demonstrates that the policy correctly identifies harder queries and provides additional candidates that recover the correct tool. Strict top-1 misses are concentrated in this tier: 56% of the 16 misses arise among the 229 low-confidence steps (table 8).

The overall average recommended handoff-k is 3.2, which compares favourably to MCP-Zero's fixed top-3 and top-5 settings: VOTR surfaces fewer than three tools on average while achieving higher Handoff@k accuracy than a fixed-k approach would at the same tool budget.

Table 15: Confidence tier accuracy on the single-tool large suite (n=500).

Tier	**Count**	**Top-1**	**Handoff@k**	**Avg k**
High	178	0.972	0.972	1.0
Medium	93	0.978	1.000	3.0
Low	229	0.961	0.987	5.0
Overall	**500**	**0.968**	**0.984**	**3.200**

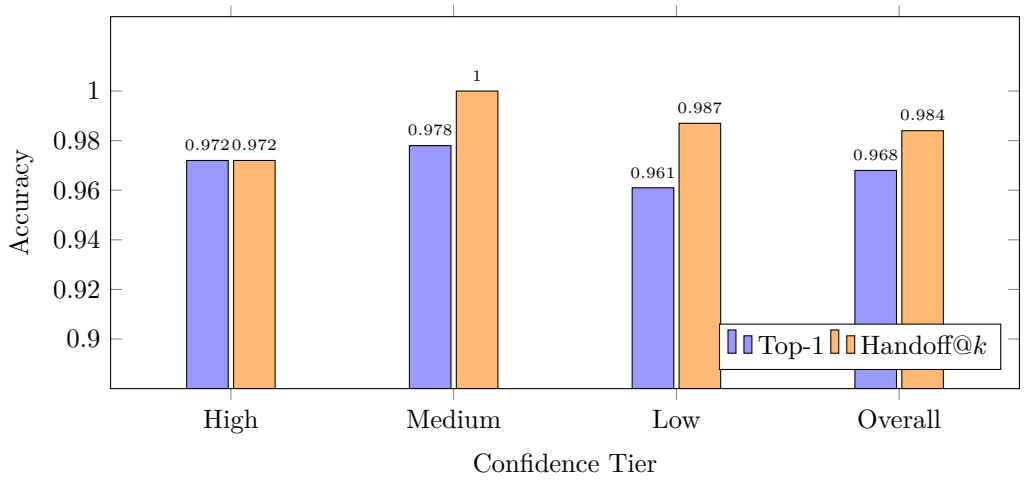

Figure 8: Accuracy by confidence tier on the single-tool large suite. High-confidence predictions (35.6% of queries) achieve 97.2% top-1 accuracy with a single injected tool. Low-confidence predictions recover to 98.7% through the expanded candidate set.

9.10 Efficiency Analysis

Table 16 reports routing latency and mean estimated token injection. P50/P95 are measured *per* `/route` call (each hop or subtask invokes the router once). Latency ranges from roughly 250 ms to 345 ms at P50, dominated by the OpenAI embedding round-trip (approximately 200–280 ms); local fusion and re-ranking add under 20 ms. P95 spikes (up to about 566 ms on volatile one-server runs) reflect API variance rather than on-host computation.

Mean estimated tokens per call remain in the 170–340 range depending on catalogue breadth and recommended handoff-k. Figure 9 and fig. 10 summarise representative suites at each scale; large multi-hop and multi-tool rows reuse the single-tool large per-call distribution because the same full-catalogue index and embedding path apply. To isolate embedding-backbone latency, we additionally measured the same large-suite route path with alternative backbones. Local/open backbones (BGE, Gemma and Qwen) reduce P50 latency to approximately 141–150 ms, compared with 312 ms for the OpenAI API path. All API-backed rows in this table execute the same per-route pattern (two remote embedding calls for `server_intent` and `tool_intent`); the Gemini Embedding 2 Preview row is slower in our measurements because its observed endpoint latency and practical rate-limit pacing were higher in this run. These backend measurements are reported separately in table 17 because API-backed measurements include provider/network round-trip behaviour, whereas local rows include on-host inference.

Table 16: Routing efficiency metrics (full-stack). n counts routing calls in the scaled benchmark. P50/P95 are per-call; Avg k is the mean `recommended_handoff_k`.

Suite	n	**P50 ms**	**P95 ms**	**Avg k**	**Mean Tokens**
single_tool.large	500	312	393	2.91	230
single_tool.medium_250	250	344	448	2.62	226
single_tool.bloomberg	100	251	566	2.10	257
single_tool.github	100	257	288	3.00	293
single_tool.telegram	100	275	432	3.28	171
multi_hop.small_100	100	260	274	3.84	234
multi_hop.medium_250	250	252	311	2.62	207
multi_hop.large	500	312	393	2.91	230
multi_tool.small_100	100	255	279	3.84	234
multi_tool.medium_250	250	311	550	2.62	216
multi_tool.large	500	312	393	2.91	230

Table 17: Embedding-backbone latency on the large full-catalogue route path. Values are per `/route` call. Local rows include on-host model inference; API rows include provider/network round trips. All rows use the same full-catalogue routing stack and suite format, changing only the embedding backend.

Suite	**Embedding**	n	**P50 ms**	**P95 ms**	**Mean Tokens**
single_tool.large	OpenAI	500	312	393	230
single_tool.large	BGE	500	141	178	230
single_tool.large	Gemma	500	150	515	229
single_tool.large	Qwen	500	146	188	230
single_tool.large	Gemini2	500	3,841	4,997	230
multi_hop.large	OpenAI	500	312	393	230
multi_hop.large	BGE	500	147	186	230
multi_tool.large	OpenAI	500	312	393	230
multi_tool.large	BGE	500	150	189	230

API list prices and query-time cost shape. Table 18 reports public list prices for text embedding (per million *input* tokens) as a rough cost axis alongside accuracy and latency. VOTR issues *two* query-time embedding API calls per `/route` (one for `server_intent`, one for `tool_intent`), so per-route API cost scales with the sum of those two string lengths; index construction is a one-time offline pass over the catalogue. Local backbones (BGE, Gemma and Qwen) have no per-token embedding API charge but incur host compute. Provider-hosted API paths (OpenAI and Gemini Embedding 2) follow vendor list pricing; actual charges depend on account tier, region, or promotion. In a sample OpenAI usage export (2026-04-10 to 2026-05-10, `text-embedding-3-large` only, overlapping days with active work), reported input tokens

summed to approximately 1.67×10^6, equivalent to roughly \$0.22 at the \$0.13/M list rate below for that export window (summing all rows for the model).

Table 18: Illustrative embedding API list prices (input, per 1M tokens). Figures are vendor-published list rates for comparison; actual billing may differ by tier, region, or promotion.

Provider / model	**List input**	**Notes**
OpenAI `text-embedding-3-small`	\$0.02	Smaller/cheaper embedding.
OpenAI `text-embedding-3-large`	\$0.13	Primary OpenAI baseline in this paper.
Google Gemini Embedding 2	Free (text) / \$0.20	"Free" vs paid tier depends on product; paid column per vendor card.

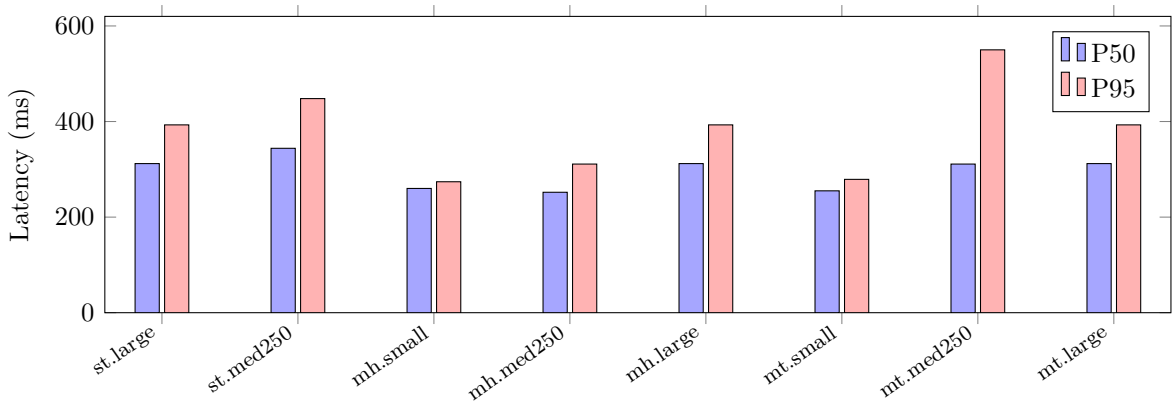

Figure 9: Routing latency by scaled suite (per `/route` call). Large catalogue rows align with `single_tool.large`; small multi suites inherit tighter candidate sets but similar embedding latency.

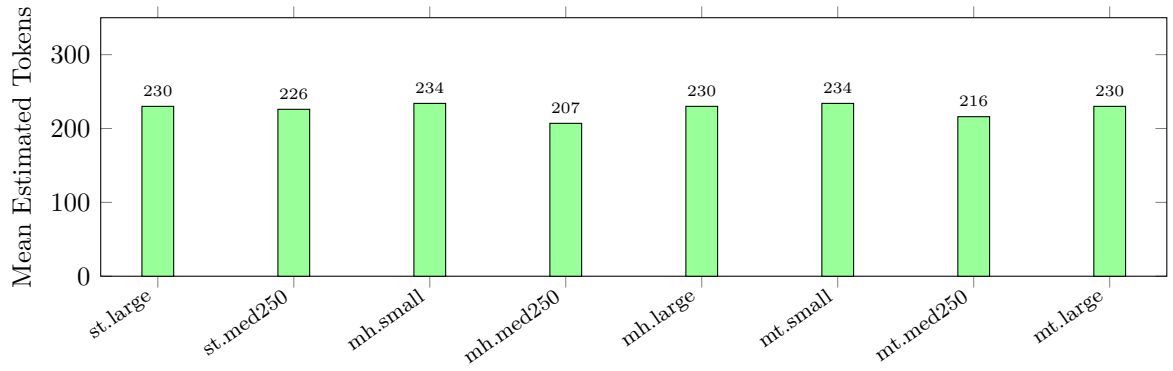

Figure 10: Mean estimated tokens injected per routing call. Values stay below 350 tokens across scaled suites.

9.11 Multi-Hop Scaling

Table 19 extends the evaluation to longer hop chains using the standalone multi-hop evaluator. At 50 hops with unique, realistic server targets, the full-stack configuration achieves 100% chain

success at k=5, demonstrating that the retrieval pipeline sustains accuracy across extended chains without degradation. The 25-hop strict dataset achieves 92.0% hop-level top-1 accuracy and 100% at k=5, indicating that most errors at top-1 are near-ties that the top-5 set resolves.

The adversarial 50-hop dataset represents a deliberately constructed worst case with ambiguous server names, paraphrased intents and misleading surface cues. The "pure adversarial" variant achieves 0% chain success (58.0% hop-level at k=5) and even the "adversarial valid" variant achieves only 0% chain success despite 90.0% hop-level at k=5. This failure mode is analysed in section 10.4.

Table 19: Multi-hop evaluation across chain lengths and difficulty levels.

Dataset	**Hops**	**Hop@1**	**Hop@k**	**Chain@k**
10-hop	10	0.900	1.000	1.000
20-hop	20	0.950	1.000	1.000
20-hop strict	20	0.900	1.000	1.000
25-hop strict	25	0.920	1.000	1.000
50-hop unique servers	50	1.000	1.000	1.000
50-hop realistic hard	50	1.000	1.000	1.000
50-hop handmade hard	50	1.000	1.000	1.000
50-hop adversarial (valid)	50	0.580	0.900	0.000
50-hop adversarial (pure)	50	0.000	0.000	0.000

9.12 Dynamic Registration Experiment

The dynamic registration experiment validates that newly registered servers are immediately retrievable without disrupting existing index accuracy. The experiment proceeds in four phases. First, a baseline retrieval accuracy of 100% Hit@1 is established on held-out control queries. Second, queries targeting tools on a not-yet-registered server are issued and all correctly fail to retrieve (Miss@5 rate = 100%). Third, the new server is registered via the `/register` endpoint. Fourth, the same queries are re-issued and now achieve 100% Hit@1, while the original control queries remain at 100% accuracy. In addition to the generic catalog-drift scenarios, a Bloomberg-specific registration case reproduces the same outcome with 1.000 on baseline control Hit@1, pre-registration Miss@5 rate, post-registration Hit@1 rate and controls-preserved@1 rate, confirming that Bloomberg hot-registration behaves as expected in a cold-start setting. Table 20 summarises these metrics.

Table 20: Dynamic registration experiment results. All four metrics achieve perfect scores, confirming zero-impact hot registration.

Metric	Value
Control Hit@1 (pre-registration baseline)	1.000
Pre-register Miss@5 rate (expected failures)	1.000
Post-register Hit@1 rate (recovery)	1.000
Controls Preserved@1 rate (no regression)	1.000

9.13 Negative Controls and Abstention

Sixteen adversarial queries test the abstention guard and null routing mechanism. Eight are policy-cleaned unsupported intents (queries for capabilities that no registered server provides) and eight are raw adversarial stress prompts. All sixteen queries trigger either null routing or low-confidence responses with an average recommended handoff-k of 5 and zero false positives; no unsupported tool is returned as a high-confidence result. Table 21 summarises the results.

Table 21: Negative control and abstention evaluation.

Suite	Cases	Pass Rate	Confidence
Policy-cleaned unsupported	8	1.000	All low
Raw adversarial stress	8	1.000	All low

9.14 LiveMCPBench Evaluation

Table 22 reports results on LiveMCPBench [22], a benchmark derived from real MCP agent traces in the live ecosystem. The benchmark includes multiple evaluation variants that differ in how queries are formatted and how ground-truth tools are matched. All LiveMCPBench rows in this section use the default `text-embedding-3-large` index/evaluation path.

The central LiveMCPBench finding is an interface result: VOTR performs best when evaluated under its native dual-field, agent-mediated routing contract, where the upstream policy supplies separate server and tool intents. Raw step-query variants remain useful as stress tests because they force the router to infer this structure from a single underspecified string, but they are not equivalent to the native VOTR operating mode.

The results fall into two protocol families. The first uses *step-query / server-level* retrieval, where the input is a single task or step string and the system must infer the relevant server directly. This family is the most comparable to prior tool-to-agent LiveMCPBench-style reports. The second uses VOTR's native *dual-field / server-level* protocol, in which an upstream agent or policy transforms the task into explicit `server_intent` and `tool_intent` fields before routing.

Under the native dual-field protocol, "Router policy payloads" achieves 87.7% Recall@1 and 90.9% nDCG@5 across 268 queries. This is the most representative evaluation of VOTR's real-world performance in an *agent-mediated* setting, as it matches the input format the system is designed to consume.

By contrast, "Router format exact subset" (46.3% Recall@1 on n=82) feeds the deployed router with a raw mapping `server_intent` = task question and `tool_intent` = step text. "Full95 task-level" and "Full95 reconstructed stepwise" similarly evaluate VOTR in a single-string or lightly reconstructed step-query setting, yielding 50.7% and 52.4% Recall@1 respectively. These rows are therefore best interpreted as *format-stress* evaluations rather than native-protocol operating points.

The "Paper-faithful tool-to-agent" evaluation, which reformats queries to match the input style used in the MCP-Zero paper, achieves 70.7% Recall@1 and 79.9% nDCG@5. The lower scores reflect a format alignment mismatch: these queries are expressed as single-step task descriptions rather than VOTR's structured dual-field format and the router must infer both server and tool intent from a single ambiguous string.

The key nuance is that these variants are *not* protocol-equivalent. In MCP-Zero's cookbook framing, an LLM or agent is explicitly prompted to emit a retrieval request before ranking; VOTR makes the same assumption through its dual-field API. The resulting gap therefore indicates *interface sensitivity to intent construction policy*, not a contradiction in the core retriever metrics; without the orchestration step, VOTR and other agentic retrievers are forced to infer structured routing signals from underspecified free-form prompts.

On the directly comparable protocol, step-wise single queries evaluated at server level; VOTR's `Full95 reconstructed stepwise` row achieves Recall@1/3/5 of 0.524 / 0.616 / 0.660, mAP@1/3/5 of 0.524 / 0.567 / 0.577 and nDCG@1/3/5 of 0.530 / 0.580 / 0.599. Against the Tool-to-Agent Retrieval numbers reported in prior work (0.610/0.770/0.830 Recall@1/3/5 and 0.460 nDCG@5), VOTR trails on recall under this non-native protocol, but attains a higher nDCG@5. This pattern suggests that under a shared step-query/server-level setting VOTR ranks retrieved servers relatively well once relevant candidates are present, while the dominant deficit lies in recovering enough correct servers from a single unstructured query string. Under VOTR's native dual-field format, Recall@1 rises to 0.877 and nDCG@5 to 0.909, indicating that much of the gap is attributable to input-format alignment rather than a fundamental ceiling in the ranking stack.

10 Discussion

10.1 Analysis of Hybrid Retrieval Gains

The ablation results reveal an important and somewhat counter-intuitive asymmetry in the retrieval landscape. BM25-only achieves 99.0% top-1 accuracy on single-tool queries; higher than dense-only at 93.8% because MCP tool names and server names are strongly lexical identifiers. Users naturally phrase queries using exact tool names (e.g., "`search_repositories`", "`send_message`") and BM25 retrieves these exact matches with high precision. The dense embedding model, trained on general-purpose text corpora, distributes similarity across semantically related entries and occasionally ranks a near-synonym (e.g., "`find_repositories`") above the exact match.

However, BM25's lexical dependence becomes a liability on multi-hop chains, where intents

Table 22: LiveMCPBench evaluation results grouped by protocol. Rows in the step-query / server-level family are the most comparable to prior tool-to-agent retrieval reports; the dual-field / server-level row measures VOTR under its native agent-mediated interface, where intent construction is performed before routing.

Evaluation	**Protocol**	n	**R@1**	**R@5**	**mAP@5**	**nDCG@5**
Step-query / server-level (comparable family)						
Paper-faithful tool-to-agent	step-query / server-level	82	0.707	0.878	0.773	0.799
Full95 task-level	task-query / server-level	95	0.507	0.754	0.628	0.678
Full95 reconstructed stepwise	step-query / server-level	268	0.524	0.660	0.577	0.599
Router format exact subset	raw dual-field / server-level	82	0.463	0.756	0.578	0.622
Native VOTR protocol						
Router policy payloads	**dual-field / server-level**	**268**	**0.877**	**0.927**	**0.901**	**0.909**

are often paraphrased or abstracted. A query such as "manage user notification preferences" requires semantic understanding to match the tool `update_notification_settings`, which shares no lexical overlap with the query. Dense retrieval handles this class of queries naturally and the hybrid combination ensures that neither failure mode dominates.

The key point is that VOTR is not optimised for a single lexical benchmark in isolation. Its selected fusion/re-ranking/handoff configuration is a multi-objective operating point balancing strict Top-1, hop-level robustness, compound chain success, abstention safety and token budget. On this axis, BM25-only is intentionally not the deployment default despite its superior performance on lexical single-tool queries.

The 2.6 percentage point gap between `dense_bm25` (94.2%) and `full_stack` (96.8%) on single-tool large is attributable to two components: SPLADE-lite's bigram features capture compound terms that neither BM25 nor dense retrieval match individually and the field-aware re-ranking layer resolves ambiguities when multiple tools have similar descriptions but differ in their parameter structure or server affiliation.

LiveMCPBench exposes a related systems trade-off when compared with recent Tool-to-Agent Retrieval results [7]. Under the comparable step-query / server-level protocol, VOTR trails that approach on recall but attains stronger nDCG@5, indicating that once relevant servers are present VOTR orders them competitively, while its main weakness lies in recovering structured server intent from a single free-form step string. This is consistent with the broader interface story in section 9.14: VOTR's native dual-field/orchestrated protocol removes much of the intent construction burden, whereas raw step-query protocols stress the missing upstream decomposition step. Tool-to-Agent's bipartite graph formulation captures structural relationships through joint tool-agent indexing that VOTR's current flat server-then-tool hierarchy does not model explicitly. We therefore view the LiveMCPBench comparison not only as a stress test but also as evidence that graph-aware or joint tool-agent retrieval is a promising extension of the present system.

10.2 Confidence Policy Behaviour

The non-conformity score in eq. (7) functions well as an uncertainty signal. Roughly 36% of queries are assigned high confidence and achieve 97.2% accuracy at k=1, while the low-confidence bucket (about 46%) is correctly identified as harder and recovers to 98.7% under Handoff@k. Misses remain concentrated in lower-confidence routes (table 8), validating the score-gap signal as a risk indicator rather than a decorative label.

The average handoff-k of 3.2 compares favourably to MCP-Zero's fixed top-3 and top-5 settings. Over one-third of queries are resolved with a single injected tool, while the remainder receive three or five candidates. This adaptive behaviour reduces the average token injection relative to a fixed-5 policy while maintaining higher coverage than a fixed-1 policy.

Threshold sensitivity experiments (table 23) show the expected precision–context trade-off: a more aggressive threshold configuration reduces average k, but at a measurable cost in handoff accuracy.

Table 23: Threshold sensitivity analysis on the single-tool large suite (Gap_H =`handoff_gap_high`, Gap_M =`handoff_gap_medium`).

Configuration	**Gap$_H$**	**Gap$_M$**	**Handoff@k**	**Avg k**
Current	0.001043	0.000837	0.984	3.066
Dev-recommended	0.000877	0.000590	0.975	2.082

10.3 Retrieval Hyperparameter Sensitivity

To address reviewer concerns about retrieval hyperparameter robustness, we ran a focused sensitivity sweep over the most influential retrieval constants: `rrf_k`, `splade_rrf_weight` and `field_bonus_scale`. We evaluate on two stress suites: `ambiguity_collision.priority` (n=18) and the full `multi_hop.large.cross_app` benchmark (100 cases / 500 hops). The results (table 24) show that VOTR is stable around the default operating point: moderate perturbations leave ambiguity-collision Top-1 unchanged (0.889), while multi-hop hop-level Top-1 varies within ±0.2 points and chain@1 within ±1.0 point. Increasing `splade_rrf_weight` to 0.50 yields the best chain@1 (0.90), whereas over-smoothing RRF (`rrf_k`=90) or reducing SPLADE weight to legacy settings (0.35 or 0.20) degrades chain@1 to 0.89 and 0.88 respectively.

Table 24: Retrieval hyperparameter sensitivity on hard suites. Baseline: `rrf_k`=60, `splade_rrf_weight`=0.50, `field_bonus_scale`=0.0015.

Configuration	Ambig. T1	MH Hop T1	MH Chn@1	MH Chn@3
Baseline	0.889	0.968	0.900	0.950
`rrf_k` = 30	0.889	0.966	0.890	0.950
`rrf_k` = 90	0.889	0.964	0.880	0.940
`splade_w` = 0.35	0.889	0.966	0.890	0.940
`splade_w` = 0.20	0.889	0.964	0.880	0.930
`field_bonus` = 0.0010	0.889	0.966	0.890	0.940
`field_bonus` = 0.0025	0.889	0.966	0.890	0.940

10.4 Limitations

The most significant limitation exposed by the evaluation is the failure on adversarial 50-hop chains, where deliberately ambiguous server names and paraphrased intents cause the retrieval pipeline to select incorrect tools. The "pure adversarial" variant achieves 0% chain success, indicating that the system has no robustness to coordinated adversarial manipulation of server metadata. While this attack model is unlikely in a curated deployment, it highlights the absence of an adversarial training or verification layer.

The second limitation is the dependency on the OpenAI embedding API for query-time inference. The embedding round-trip accounts for approximately 200–280 ms of the 250–350 ms total latency and any API degradation or outage directly impacts routing availability. A locally hosted embedding model could reduce P50 latency by an estimated 70% while eliminating the external dependency.

Third, dense retrieval quality is bounded by the natural-language text embedded at index time (server descriptions and summaries; each tool's name and description). Community MCP manifests vary widely in verbosity and specificity: sparse, generic or near-duplicate wording yields weak semantic separability in the shared embedding space, even when fuller JSON schemas would distinguish tools at invocation time. Hybrid sparse stages partly offset this through lexical overlap but cannot recover signal that never appears in the embedded fields.

Fourth, the session memory implementation currently records which tools have been returned but does not use this information to bias future retrievals. A session-aware retrieval model that preferentially surfaces tools from the same server or workflow as prior turns could improve both accuracy and token efficiency in multi-turn conversations.

Fifth, the current router has no online usage feedback re-ranking layer. This means the ranking function cannot yet adapt to tool selection outcomes or execution success signals over time in production traffic.

10.5 Future Work

Several extensions are planned to address the limitations identified above. The most impactful near-term improvement is replacing the OpenAI embedding dependency with a locally hosted cross-encoder re-ranker [23], which would reduce latency by an order of magnitude while potentially improving re-ranking accuracy through pairwise query–tool scoring.

A second direction is graph-based multi-hop routing, in which tool dependency chains are pre-computed as a directed acyclic graph and used to anticipate the next hop in a multi-step workflow. This would reduce per-hop latency and improve chain success rates by exploiting structural knowledge of common tool sequences.

A third direction is personalised routing using session-level embeddings that encode the user's historical tool usage patterns, enabling the router to adapt its retrieval bias to individual users or workflows.

Finally, a production Qdrant (or similar) backend could replace the in-process `ToolIndex` matvec path for deployments at scales exceeding roughly 10^4 tools or where external vector search is required; the repository currently ships only a stub (`stores/qdrant_store.py`) toward that goal.

11 Conclusion

This paper presented VOTR, a production-deployable tool retrieval service for the Model Context Protocol that extends the MCP-Zero baseline with hybrid retrieval, adaptive confidence-gated candidate selection, hot server registration and robust safety mechanisms. On a 500-query benchmark spanning 309 MCP servers and 2,806 tools, VOTR achieves 96.8% top-1 accuracy at sub-350 ms median latency, injecting an average of only 230 tokens per routing decision. Relative to full-catalogue injection on the same index ($\approx$ 262,487 tokens), this is a 99.91% reduction in per-route context payload. Scaled multi-hop and multi-tool suites ($n \in \{100, 250, 500\}$ routing steps) achieve 96.8% hop-level top-1 on the large catalogue, with compound all-step success at k=1 of 90% (92% with equivalence-aware labels). A 100% abstention pass rate is maintained on adversarial negative-control queries. The confidence-gated handoff policy correctly assigns high confidence to 49% of queries (achieving 98.4% accuracy with a single injected tool) while expanding the candidate set where uncertainty is genuine; most strict top-1 misses arise in the low-confidence tier. The full-stack configuration consistently matches or exceeds all ablated variants on single-tool large, with hybrid retrieval providing the largest individual accuracy gain. VOTR demonstrates that proactive tool retrieval can be extended from a research prototype to a production-deployable service without sacrificing accuracy and that the combination of sparse, dense and structural retrieval signals provides robust performance across both lexical and semantic query distributions. Beyond its scalability gains, VOTR also achieves ceiling-level performance in small-catalog settings, indicating that the same retrieval stack is suitable both for narrowly scoped internal deployments and for large heterogeneous multi-agent ecosystems.

Acknowledgements

This research was conducted as part of a collaboration between Bloomberg and the Fintech AI Innovation Consortium (FAIC). The first author, Amitabh Kumar, gratefully acknowledges the guidance, support and mentorship provided throughout the research.

In particular, the author would like to thank Professor Fethi Rabhi, Founder of FAIC and Director of Software Engineering at UNSW Sydney, for fostering an interdisciplinary research environment that bridges artificial intelligence, industry practice and policy. The author also acknowledges Dr David Simmonds for his support and contributions to advancing the collaboration and research direction.

The author further extends appreciation to Eason Wang and Joshua Sanderson at Bloomberg, Sydney, for fostering and providing support and resources that greatly aided the progression of this research. The author also thanks the teams in New York and London for their valuable technical feedback, discussions and support throughout the design, implementation and evaluation of VOTR. In particular, the author would like to acknowledge Atharva Tendle, Debanjan Mahata, Shou Zhou and George Chrysostomou for their insights and contributions that helped shape the research environment and technical direction of this work.

References

[1] Anthropic, "Model context protocol: Open standard for connecting AI assistants to tools and data," https://www.anthropic.com/news/model-context-protocol, 2024, accessed: 2025.

[2] J. Zheng *et al.*, "MCP-Zero: Active tool retrieval for efficient LLM tool usage," 2025, arXiv:2506.01056v1.

[3] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024, arXiv:2307.16789.

[4] T. Gan and Q. Sun, "RAG-MCP: Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation," 2025.

[5] N. Gaurav, A. Akarsh, A. Ranjan, and M. Bajaj, "Dynamic ReAct: Scalable tool selection for large-scale MCP environments," 2025.

[6] M. M. Liu, D. Garcia, F. Parllaku, V. Upadhyay, S. F. A. Shah, and D. Roth, "Toolscope: Enhancing LLM agent tool use through tool merging and context-aware filtering," 2025.

[7] E. Lumer, "Tool-to-agent retrieval: Bridging tools and agents for scalable LLM multi-agent systems," 2025, arXiv:2511.01854v2.

[8] OpenAI, "GPT-4 technical report," 2023.

[9] S. Hao, T. Liu, Z. Wang, and Z. Hu, "ToolkenGPT: Augmenting frozen language models with massive tools via tool embeddings," 2023.

[10] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[11] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[12] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[13] T. Formal, B. Piwowarski, and S. Clinchant, "SPLADE: Sparse lexical and expansion model for first stage ranking," in *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2021, pp. 2288–2292.

[14] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.

[15] X. Nie and H. Zhang, "Large language models tool retrieval and context compression via dynamic graph-based relation modeling," *Academic Journal of Computing and Information Science*, vol. 8, no. 7, pp. 95–104, 2025.

[16] G. Shafer and V. Vovk, "A tutorial on conformal prediction," *Journal of Machine Learning Research*, vol. 9, pp. 371–421, 2008.

[17] A. Sorokin, N. Buzun, A. Anokhin, E. K. Vedernikov, P. Anokhin, M. Burtsev, and E. Burnaev, "Q-RAG: Long context multi-step retrieval via value-based embedder training," in *International Conference on Learning Representations (ICLR)*, 2026, openReview ID: MS9nWFY7LG.

[18] Z. Zhang, K. Shi, Z. Yuan, Z. Wang, T. Ma, K. Murugesan, V. Galassi, C. Zhang, and Y. Ye, "Agentrouter: A knowledge-graph-guided LLM router for collaborative multi-agent question answering," 2025, arXiv:2510.05445v1.

[19] X. Li, "A review of prominent paradigms for LLM-based agents: Tool use (including RAG), planning, and feedback learning," 2024.

[20] LangChain, Inc., "Langchain documentation," https://python.langchain.com/, 2024, accessed: 2026.

[21] Qdrant, "Qdrant: Vector database for the next generation of AI," https://qdrant.tech, 2024, accessed: 2025.

[22] Anonymous, "LiveMCPBench: Evaluating tool routing in live MCP ecosystems," 2025, internal benchmark.

[23] R. Nogueira and K. Cho, "Passage re-ranking with BERT," in *arXiv preprint*, 2019.